Chapter 11 ▪ User Interface Components with Swing

The previous chapter was written primarily to show you how to use the event model in Java. In the process, you took the first steps toward learning how to build a graphical user interface. This chapter shows you the most important tools you'll need to build more full-featured GUIs.

We start out with a tour of the architectural underpinnings of Swing. Knowing what goes on "under the hood" is important in understanding how to use some of the more advanced components effectively. I then show you the most common user interface components in Swing, such as text fields, radio buttons, and menus. Next, you will learn how to use layout managers to arrange these components. Finally, you'll see how to implement dialog boxes in Swing.

This chapter covers the basic Swing components such as text components, buttons, and sliders. These are the essential user interface components that you will need most frequently. We will cover advanced Swing components in **Chapter 12**.

11.1. Swing and the Model-View-Controller Design Pattern

Let's step back for a minute and think about the pieces that make up a user interface component such as a button, a checkbox, a text field, or a sophisticated tree control. Every component has three characteristics:

- Its *content*, such as the state of a button (pushed in or not), or the text in a text field
- Its *visual appearance* (color, size, and so on)
- Its *behavior* (reaction to events)

Even a seemingly simple component such as a button exhibits some moderately complex interaction among these characteristics. Obviously, the visual appearance of a button depends on the look-and-feel. A Metal button looks different from a Windows button or a Motif button. In addition, the appearance depends on the button state; when a button is pushed in, it needs to be redrawn to look different. The state depends on the events that the button receives. When the user depresses the mouse inside the button, the button is pushed in.

Of course, when you use a button in your programs, you simply consider it as a *button*; you don't think too much about the inner workings and characteristics. That, after all, is the job of the programmer who implemented the button. However, programmers who implement buttons and all other user interface components are motivated to think a little harder about them, so that they work well no matter what look-and-feel is in effect.

To do this, the Swing designers turned to a well-known design pattern: the *model-view-controller* (MVC) pattern. This design pattern tells us to provide three separate objects:

- The *model,* which stores the content
- The *view,* which displays the content
- The *controller*, which handles user input

The pattern specifies precisely how these three objects interact. The model stores the content and has *no user interface*. For a button, the content is pretty trivial—just a small set of flags that tells whether the button is currently pushed in or out, whether it is active or inactive, and so on. For a text field, the content is a bit more interesting. It

is a string object that holds the current text. This is *not the same* as the view of the content—if the content is larger than the text field, the user sees only a portion of the text displayed (see **Figure 11.1**).

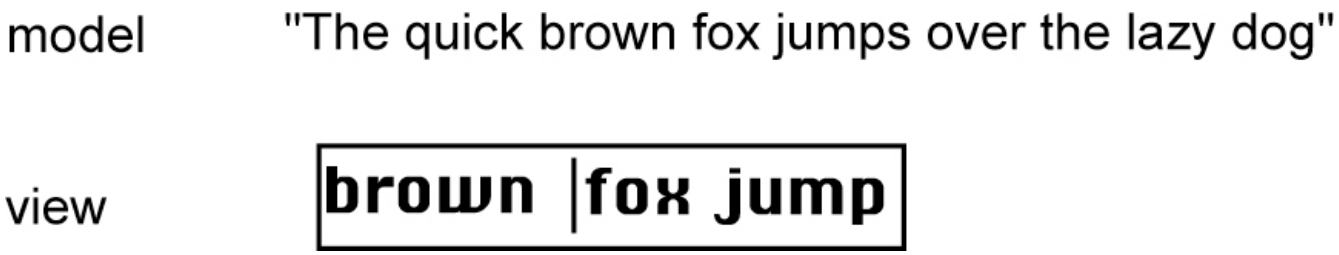


Figure 11.1: Model and view of a text field

The model must implement methods to change the content and to discover what the content is. For example, a text model has methods to add or remove characters in the current text and to return the current text as a string. Again, keep in mind that the model is completely nonvisual. It is the job of a view to draw the data stored in the model.



Note: The term "model" is perhaps unfortunate because we often think of a model as a representation of an abstract concept. Car and airplane designers build models to simulate real cars and planes. But that analogy really leads you astray when thinking about the model-view-controller pattern. In this design pattern, the model stores the complete content, and the view gives a (complete or incomplete) visual representation of the content. A better analogy might be the model who poses for an artist. It is up to the artist to look at the model and create a view. Depending on the artist, that view might be a formal portrait, an impressionist painting, or a cubist drawing with strangely contorted limbs.

One of the advantages of the model-view-controller pattern is that a model can have multiple views, each showing a different part or aspect of the full content. For example, an HTML editor can offer two *simultaneous* views of the same content: a WYSIWYG view and a "raw tag" view (see **Figure 11.2**). When the model is updated through the controller of one of the views, it tells both attached views about the change. When the views are notified, they refresh themselves automatically. Of course, for a simple user interface component such as a button, you won't have multiple views of the same model.

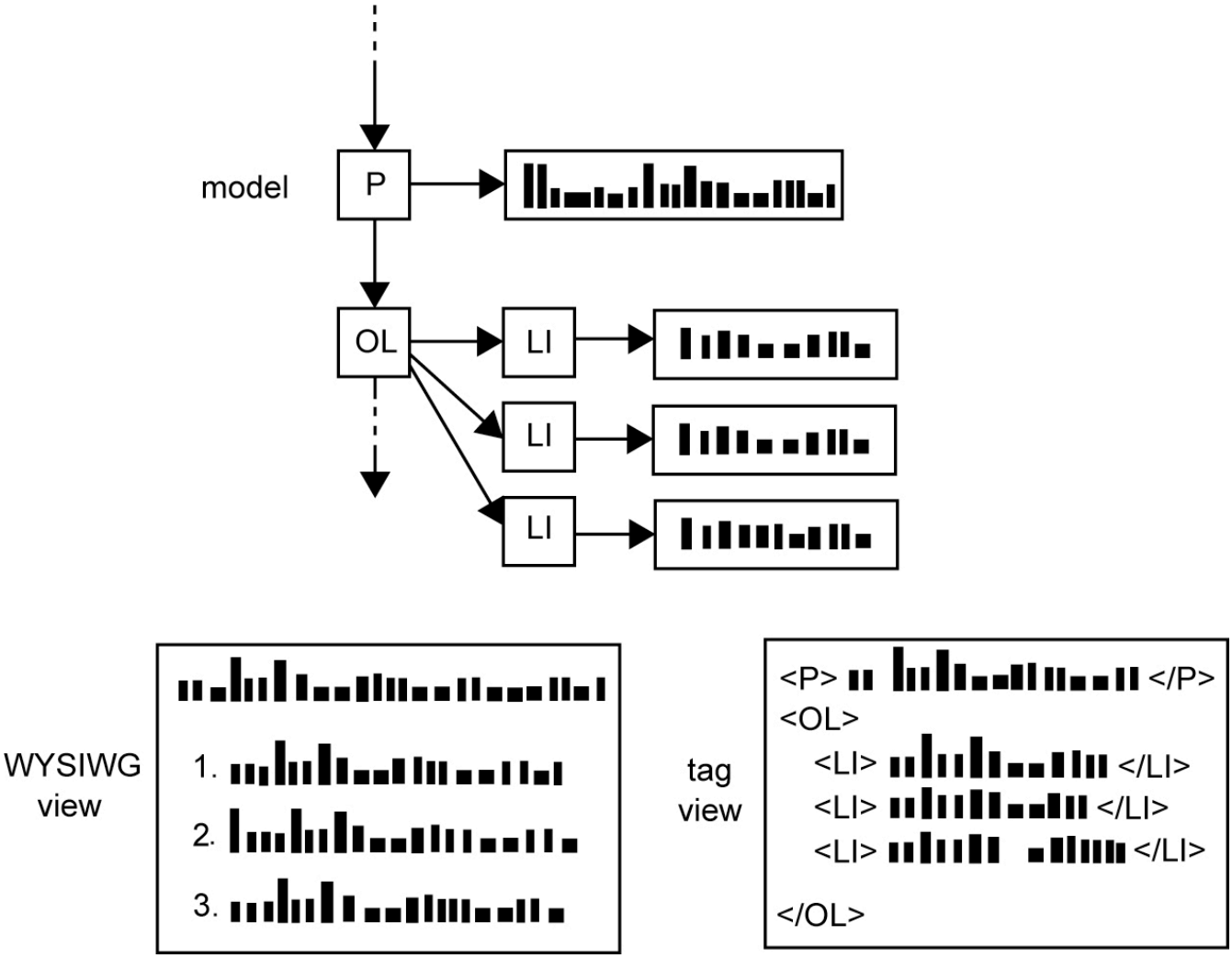


Figure 11.2: Two separate views of the same model

The controller handles the user-input events, such as mouse clicks and keystrokes. It then decides whether to translate these events into changes in the model or the view. For example, if the user presses a character key in a text box, the controller calls the “insert character” command of the model. The model then tells the view to update itself. The view never knows why the text changed. But if the user presses a cursor key, the controller may tell the view to scroll. Scrolling the view has no effect on the underlying text, so the model never knows that this event happened.

Figure 11.3 shows the interactions among model, view, and controller objects.

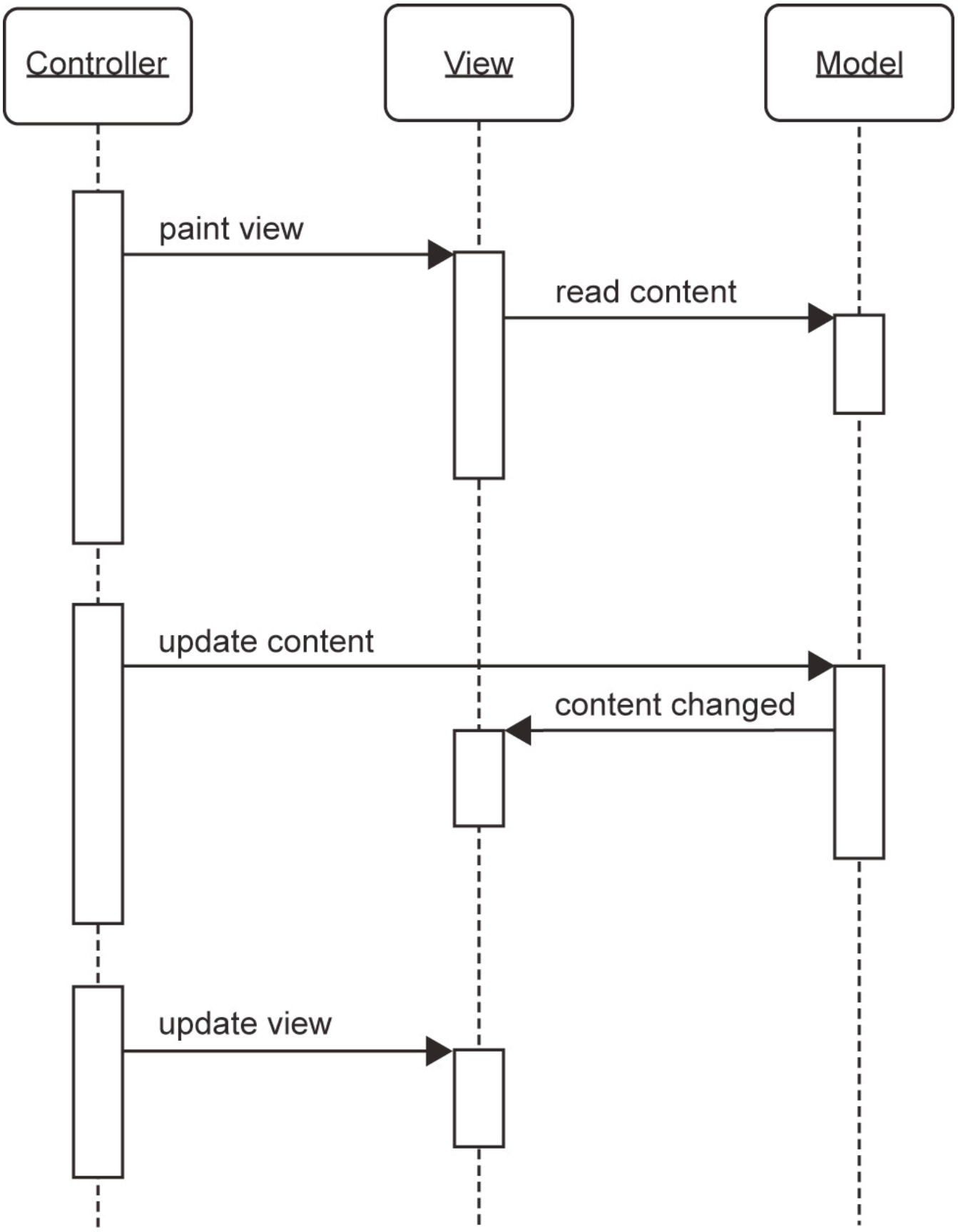


Figure 11.3: Interactions among model, view, and controller objects

For most Swing components, the model class implements an interface whose name ends in `Model`; in this case, the interface is called `ButtonModel`. Classes implementing that interface can define the state of the various kinds of buttons. Actually, buttons aren't all that complicated, and the Swing library contains a single class, called `DefaultButtonModel`, that implements this interface.

You can get a sense of the sort of data maintained by a button model by looking at the properties of the `ButtonModel` interface—see **Table 11.1**.

Table 11.1: Properties of the `ButtonModel` Interface

Property Name	Value
`actionCommand`	The action command string associated with this button
`mnemonic`	The keyboard mnemonic for this button
`armed`	`true` if the button was pressed and the mouse is still over the button
`enabled`	`true` if the button is selectable
`pressed`	`true` if the button was pressed but the mouse button hasn't yet been released
`rollover`	`true` if the mouse is over the button
`selected`	`true` if the button has been toggled on (used for checkboxes and radio buttons)

Each `JButton` object stores a button model object which you can retrieve.

```
var button = new JButton("Blue");
ButtonModel model = button.getModel();
```

In practice, you won't care—the minutiae of the button state are only of interest to the view that draws it. All the important information—such as whether a button is enabled—is available from the `JButton` class. (Of course, the `JButton` then asks its model to retrieve that information.)

Have another look at the `ButtonModel` interface to see what *isn't* there. The model does *not* store the button label or icon. There is no way to find out what's on the face of a button just by looking at its model. (Actually, as you will see in **Section 11.4.2**, this purity of design is the source of some grief for the programmer.)

It is also worth noting that the *same* model (namely, `DefaultButtonModel`) is used for push buttons, radio buttons, checkboxes, and even menu items. Of course, each of these button types has different views and controllers. When using the Metal look-and-feel, the `JButton` uses a class called `BasicButtonUI` for the view and a class called `ButtonUIListener` as controller. In general, each Swing component has an associated view object that ends in `UI`. But not all Swing components have dedicated controller objects.

So, having read this short introduction to what is going on under the hood in a `JButton`, you may be wondering: Just what is a `JButton` really? It is simply a wrapper class inheriting from `JComponent` that holds the `DefaultButtonModel` object, some view data (such as the button label and icons), and a `BasicButtonUI` object that is responsible for the button view and installs a `BasicButtonListener` with button-specific controller functionality.

11.2. Introduction to Layout Management

Before we go on to discussing individual Swing components, such as text fields and radio buttons, let's briefly cover how to arrange these components inside a frame.

Of course, Java development environments have drag-and-drop GUI builders. Nevertheless, it is important to know exactly what goes on "under the hood" because even the best of these tools will usually require hand-

tweaking.

11.2.1. Layout Managers

Let's start by reviewing the program from **Chapter 10** that used buttons to change the background color of a frame.

The buttons are contained in a `JPanel` object and are managed by the *flow layout manager*, the default layout manager for a panel. **Figure 11.4** shows what happens when you add more buttons to the panel. As you can see, a new row is started when there is no more room.

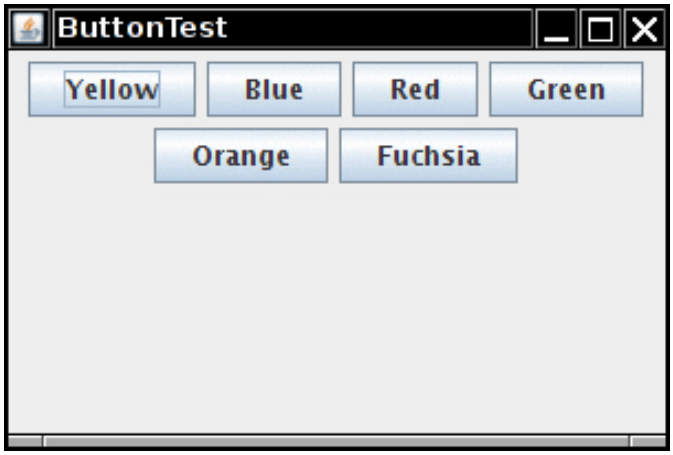


Figure 11.4: A panel with six buttons managed by a flow layout

Moreover, the buttons stay centered in the panel, even when the user resizes the frame (see **Figure 11.5**).

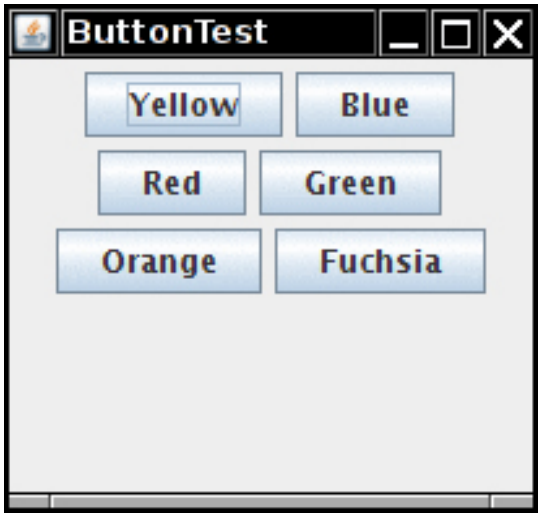


Figure 11.5: Changing the panel size rearranges the buttons automatically.

In general, *components* are placed inside *containers*, and a *layout manager* determines the positions and sizes of components in a container.

Buttons, text fields, and other user interface elements extend the class `Component`. Components can be placed inside containers, such as panels. Containers can themselves be put inside other containers, so the class `Container` extends `Component`. **Figure 11.6** shows the inheritance hierarchy for `Component`.

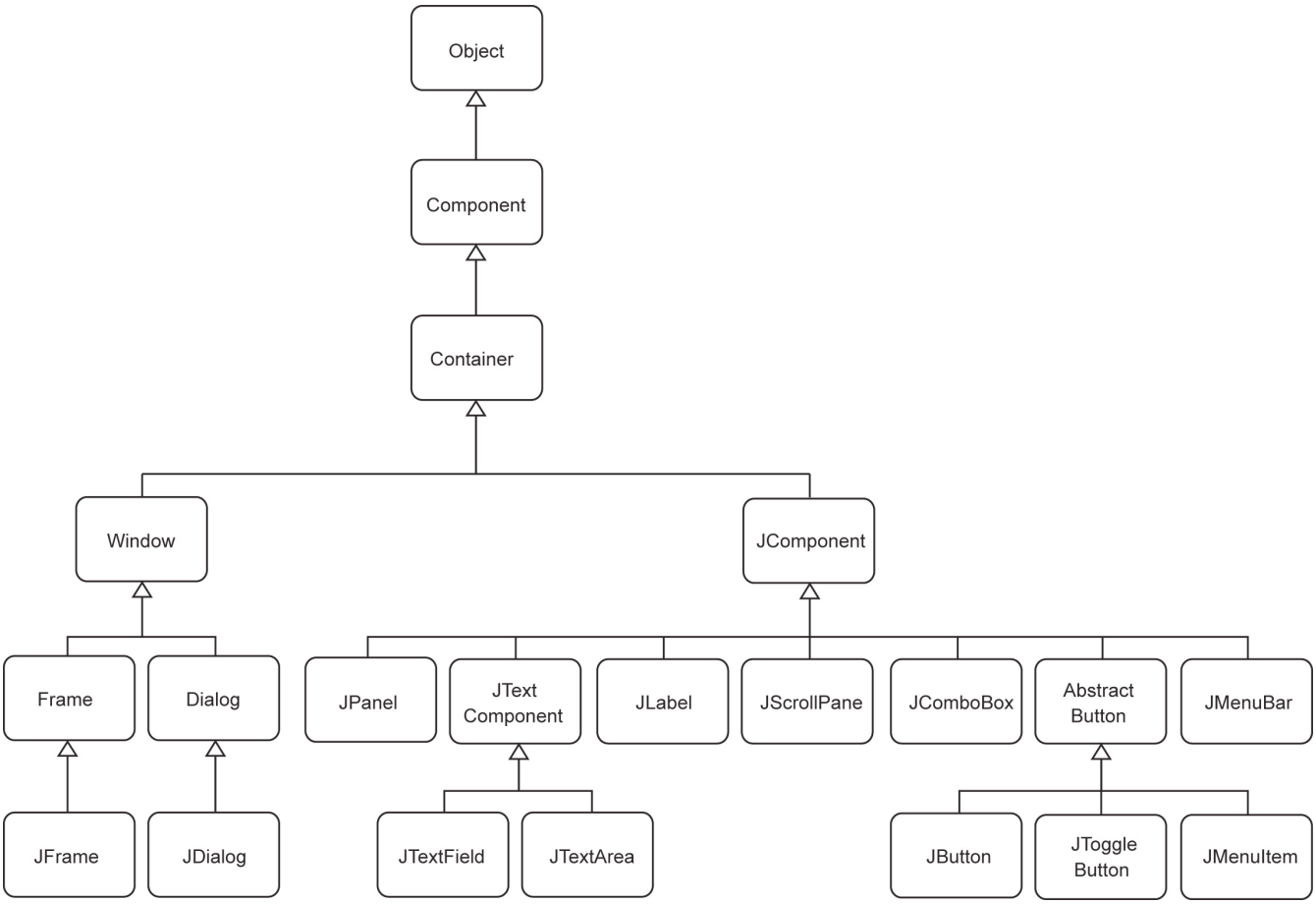


Figure 11.6: Inheritance hierarchy for the `Component` class



Note: Unfortunately, the inheritance hierarchy is somewhat unclean in two respects. First, top-level windows, such as `JFrame`, are subclasses of `Container` and hence `Component`, but they cannot be placed inside other containers. Moreover, `JComponent` is a subclass of `Container`, not `Component`. Therefore one can add other components into a `JButton`. (However, those components would not be displayed.)

Each container has a default layout manager, but you can always set your own. For example, the statement

```
panel.setLayout(new GridLayout(4, 4));
```

uses the `GridLayout` class to lay out the components in four rows and four columns. When you add components to the container, the `add` method of the container passes the component and any placement directions to the layout manager.

`java.awt.Container` 1.0

- `void setLayout(LayoutManager m)`
  sets the layout manager for this container.
- `Component add(Component c)`
- `Component add(Component c, Object constraints)` 1.1
  add a component to this container and return the component reference.

`java.awt.FlowLayout` 1.0

- `FlowLayout()`
- `FlowLayout(int align)`
- `FlowLayout(int align, int hgap, int vgap)`

construct a new `FlowLayout`. The `align` parameter is one of `LEFT`, `CENTER`, or `RIGHT`.

11.2.2. Border Layout

The *border layout manager* is the default layout manager of the content pane of every `JFrame`. Unlike the flow layout manager, which completely controls the position of each component, the border layout manager lets you choose where you want to place each component. You can choose to place the component in the center, north, south, east, or west of the content pane (see **Figure 11.7**).

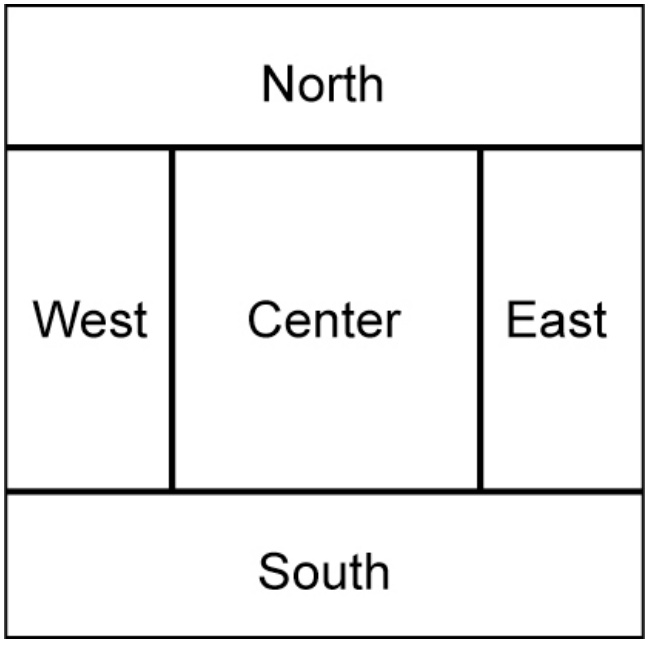


Figure 11.7: Border layout

For example:

```
frame.add(component, BorderLayout.SOUTH);
```

The edge components are laid out first, and the remaining available space is occupied by the center. When the container is resized, the dimensions of the edge components are unchanged, but the center component changes its size. Add components by specifying a constant `CENTER`, `NORTH`, `SOUTH`, `EAST`, or `WEST` of the `BorderLayout` class. Not all of the positions need to be occupied. If you don't supply any value, `CENTER` is assumed.



Note: The `BorderLayout` constants are defined as strings. For example, `BorderLayout.SOUTH` is defined as the string `"South"`. This is safer than using strings. If you accidentally misspell a string, for example, `frame.add(component, "south")`, the compiler won't catch that error.

Unlike the flow layout, the border layout grows all components to fill the available space. (The flow layout leaves each component at its preferred size.) This is a problem when you add a button:

```
frame.add(yellowButton, BorderLayout.SOUTH); // don't
```

Figure 11.8 shows what happens when you use the preceding code fragment. The button has grown to fill the entire southern region of the frame. And, if you were to add another button to the southern region, it would just displace the first button.



Figure 11.8: A single button managed by a border layout

To solve this problem, use additional panels. For example, look at Figure 11.9. The three buttons at the bottom of the screen are all contained in a panel. The panel is put into the southern region of the content pane.

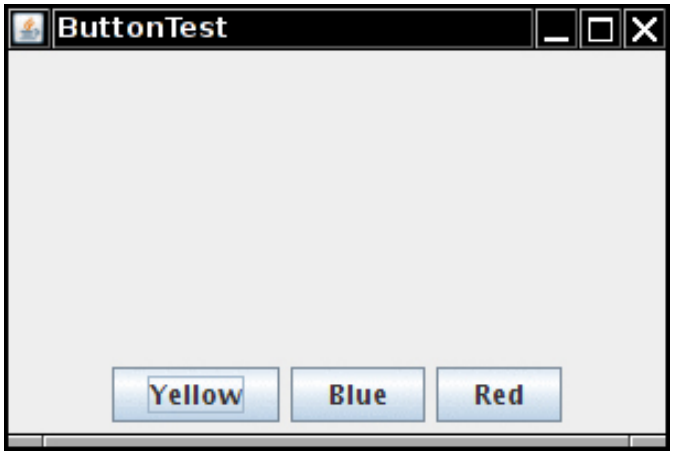


Figure 11.9: Panel placed in the southern region of the frame

To achieve this configuration, first create a new `JPanel` object, then add the individual buttons to the panel. The default layout manager for a panel is a `FlowLayout`, which is a good choice for this situation. Add the individual buttons to the panel, using the `add` method you have seen before. The position and size of the buttons is under the control of the `FlowLayout` manager. This means the buttons stay centered within the panel and do not expand to fill the entire panel area. Finally, add the panel to the content pane of the frame.

```
var panel = new JPanel();
panel.add(yellowButton);
panel.add(blueButton);
panel.add(redButton);
frame.add(panel, BorderLayout.SOUTH);
```

The border layout expands the size of the panel to fill the entire southern region.

`java.awt.BorderLayout` 1.0

- `BorderLayout()`
- `BorderLayout(int hgap, int vgap)`

  construct a new `BorderLayout`.

11.2.3. Grid Layout

The grid layout arranges all components in rows and columns like a spreadsheet. All components are given the same size. The calculator program in **Figure 11.10** uses a grid layout to arrange the calculator buttons. When you resize the window, the buttons grow and shrink, but all buttons have identical sizes.

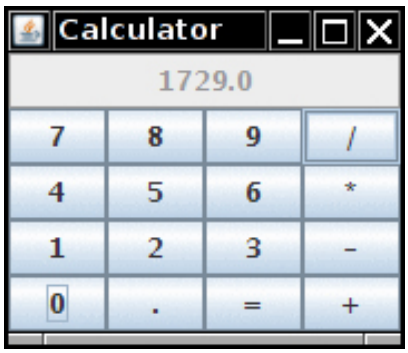


Figure 11.10: A calculator

In the constructor of the grid layout object, you specify how many rows and columns you need.

```
panel.setLayout(new GridLayout(4, 4));
```

Add the components, starting with the first entry in the first row, then the second entry in the first row, and so on.

```
panel.add(new JButton("1"));
panel.add(new JButton("2"));
```

Of course, few applications have as rigid a layout as the face of a calculator. In practice, small grids (usually with just one row or one column) can be useful to organize partial areas of a window. For example, if you want to have a row of buttons of identical sizes, you can put the buttons inside a panel that is governed by a grid layout with a single row.

`java.awt.GridLayout` 1.0

- `GridLayout(int rows, int columns)`
- `GridLayout(int rows, int columns, int hgap, int vgap)`

  construct a new `GridLayout`. One of `rows` and `columns` (but not both) may be zero, denoting an arbitrary number of components per row or column.

11.3. Text Input

We are finally ready to start introducing the Swing user interface components. Let's begin with the components that let a user input and edit text. You can use the `JTextField` and `JTextArea` components for text input. A

text field can accept only one line of text; a text area can accept multiple lines of text. A `JPasswordField` accepts one line of text without showing the contents.

All three of these classes inherit from a class called `JTextComponent`. You will not be able to construct a `JTextComponent` yourself because it is an abstract class. On the other hand, as is so often the case in Java, when you go searching through the API documentation, you may find that the methods you are looking for are actually in the parent class `JTextComponent` rather than the derived class. For example, the methods that get or set the text in a text field or text area are actually in `JTextComponent`.

`javax.swing.text.JTextComponent` 1.2

- `String getText()`
- `void setText(String text)`
  get or set the text of this text component.
- `boolean isEditable()`
- `void setEditable(boolean b)`
  get or set the `editable` property that determines whether the user can edit the content of this text component.

11.3.1. Text Fields

The usual way to add a text field to a window is to add it to a panel or other container—just as you would add a button:

```
var panel = new JPanel();
var textField = new JTextField("Default input", 20);
panel.add(textField);
```

This code adds a text field and initializes it by placing the string `"Default input"` inside it. The second parameter of this constructor sets the width. In this case, the width is 20 “columns.” Unfortunately, a column is a rather imprecise measurement. One column is the expected width of one character in the font you are using for the text. The idea is that if you expect the inputs to be *n* characters or less, you are supposed to specify *n* as the column width. In practice, this measurement doesn’t work out too well, and you should add 1 or 2 to the maximum input length to be on the safe side. Also, keep in mind that the number of columns is only a hint to the AWT that gives the *preferred* size. If the layout manager needs to grow or shrink the text field, it can adjust its size. The column width that you set in the `JTextField` constructor is not an upper limit on the number of characters the user can enter. The user can still type in longer strings, but the input scrolls when the text exceeds the length of the field. Users tend to find scrolling text fields irritating, so you should size the fields generously. If you need to reset the number of columns at runtime, you can do that with the `setColumns` method.



Tip: After changing the size of a text box with the `setColumns` method, call the `revalidate` method of the surrounding container.

```
textField.setColumns(10);
panel.revalidate();
```

The `revalidate` method recomputes the size and layout of all components in a container. After you use the `revalidate` method, the layout manager resizes the container, and the changed size of the text field will be visible.

The `revalidate` method belongs to the `JComponent` class. It doesn't immediately resize the component but merely marks it for resizing. This approach avoids repetitive calculations if multiple components request to be resized. However, if you want to recompute all components inside a `JFrame`, you have to call the `validate` method—`JFrame` doesn't extend `JComponent`.

In general, users add text (or edit an existing text) in a text field. Quite often these text fields start out blank. To make a blank text field, just don't provide a string argument for the `JTextField` constructor:

```
var textField = new JTextField(20);
```

You can change the content of the text field at any time by using the `setText` method from the `JTextComponent` parent class mentioned in the previous section. For example:

```
textField.setText("Hello!");
```

And, as was mentioned in the previous section, you can find out what the user typed by calling the `getText` method. This method returns the exact text that the user has typed. To strip any extraneous leading and trailing spaces from the data in a text field, apply the `strip` method to the return value of `getText`:

```
String text = textField.getText().strip();
```

To change the font in which the user text appears, use the `setFont` method.

`javax.swing.JTextField` **1.2**

- `JTextField(int columns)`
  constructs an empty `JTextField` with the specified number of columns.
- `JTextField(String text, int columns)`
  constructs a new `JTextField` with an initial string and the specified number of columns.
- `int getColumns()`
- `void setColumns(int columns)`
  get or set the number of columns that this text field should use.

`javax.swing.JComponent` **1.2**

- `void revalidate()`
  causes the position and size of a component to be recomputed.
- `void setFont(Font f)`
  sets the font of this component.

`java.awt.Component` **1.0**

- `void validate()`
  recomputes the position and size of a component. If the component is a container, the positions and sizes of its components are recomputed.
- `Font getFont()`
  gets the font of this component.

11.3.2. Labels and Labeling Components

Labels are components that hold text. They have no decorations (for example, no boundaries). They also do not react to user input. You can use a label to identify components. For example, unlike buttons, text fields have no label to identify them. To label a component that does not itself come with an identifier:

1. Construct a `JLabel` component with the correct text.
2. Place it close enough to the component you want to identify so that the user can see that the label identifies the correct component.

The constructor for a `JLabel` lets you specify the initial text or icon and, optionally, the alignment of the content. Use constants from the `SwingConstants` interface to specify alignment. That interface defines a number of useful constants such as `LEFT`, `RIGHT`, `CENTER`, `NORTH`, `EAST`, and so on. The `JLabel` class is one of several Swing classes that implement this interface. Therefore, you can specify a right-aligned label either as

```
var label = new JLabel("User name: ", SwingConstants.RIGHT);
```

or

```
var label = new JLabel("User name: ", JLabel.RIGHT);
```

The `setText` and `setIcon` methods let you set the text and icon of the label at runtime.



Tip: You can use both plain and HTML text in buttons, labels, and menu items. I don't recommend HTML in buttons—it interferes with the look-and-feel. But HTML in labels can be very effective. Simply surround the label string with `<html>. . .</html>`, like this:

```
label = new JLabel("<html><b>Required</b> entry:</html>");
```

Note that the first component with an HTML label may take some time to be displayed because the rather complex HTML rendering code must be loaded.

Labels can be positioned inside a container like any other component. This means you can use the techniques you have seen before to place your labels where you need them.

`javax.swing.JLabel` 1.2

- `JLabel(String text)`
- `JLabel(Icon icon)`
- `JLabel(String text, int align)`
- `JLabel(String text, Icon icon, int align)`
  construct a label. The `align` parameter is one of the `SwingConstants` constants `LEFT` (default), `CENTER`, or `RIGHT`.
- `String getText()`
- `void setText(String text)`
  get or set the text of this label.

- `Icon getIcon()`
- `void setIcon(Icon icon)`
  get or set the icon of this label.

11.3.3. Password Fields

Password fields are a special kind of text fields. To prevent nosy bystanders from seeing your password, the characters that the user enters are not actually displayed. Instead, each typed character is represented by an *echo character*, such as a bullet (`•`). Swing supplies a `JPasswordField` class that implements such a text field.

The password field is another example of the power of the model-view-controller architecture pattern. The password field uses the same model to store the data as a regular text field, but its view has been changed to display all characters as echo characters.

`javax.swing.JPasswordField` 1.2

- `JPasswordField(String text, int columns)`
  constructs a new password field.
- `void setEchoChar(char echo)`
  sets the echo character for this password field. This is advisory; a particular look-and-feel may insist on its own choice of echo character. A value of `0` resets the echo character to the default.
- `char[] getPassword()`
  returns the text contained in this password field. For stronger security, you should overwrite the content of the returned array after use. (The password is not returned as a `String` because a string would stay in the virtual machine until it is garbage-collected.)

11.3.4. Text Areas

Sometimes, you need to collect user input that is more than one line long. As mentioned earlier, you can use the `JTextArea` component for this. When you place a text area component in your program, a user can enter any number of lines of text, using the Enter key to separate them. Each line ends with a `'\n'` (even on Windows). Figure 11.11 shows a text area at work.

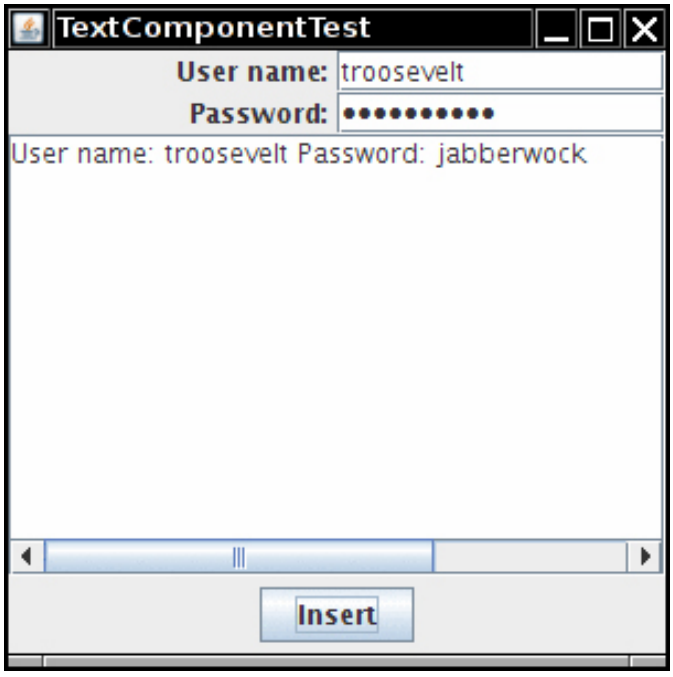


Figure 11.11: Text components

In the constructor for the `JTextArea` component, specify the number of rows and columns for the text area. For example,

```
textArea = new JTextArea(8, 40); // 8 lines of 40 columns each
```

where the `columns` parameter works as before—and you still need to add a few more columns for safety's sake. Also, as before, the user is not restricted to the number of rows and columns; the text simply scrolls when the user inputs too much. You can also use the `setColumns` method to change the number of columns and the `setRows` method to change the number of rows. These numbers only indicate the preferred size—the layout manager can still grow or shrink the text area.

If there is more text than the text area can display, the remaining text is simply clipped. You can avoid clipping long lines by turning on line wrapping:

```
textArea.setLineWrap(true); // long lines are wrapped
```

This wrapping is a visual effect only; the text in the document is not changed—no automatic `'\n'` characters are inserted into the text.

11.3.5. Scroll Panes

In Swing, a text area does not have scrollbars. If you want scrollbars, you have to place the text area inside a *scroll pane*.

```
textArea = new JTextArea(8, 40);
var scrollPane = new JScrollPane(textArea);
```

The scroll pane now manages the view of the text area. Scrollbars automatically appear if there is more text than the text area can display, and they vanish again if text is deleted and the remaining text fits inside the area. The scrolling is handled internally by the scroll pane—your program does not need to process scroll events.

This is a general mechanism that works for any component, not just text areas. To add scrollbars to a component, put them inside a scroll pane.

Listing 11.1 demonstrates the various text components. This program shows a text field, a password field, and a text area with scrollbars. The text field and password field are labeled. Click on "Insert" to insert the field contents into the text area.



Note: The `JTextArea` component displays plain text only, without special fonts or formatting. To display formatted text (such as HTML), you can use the `JEditorPane` class that is discussed in **Chapter 12**.

Listing 11.1 v2ch11/text/TextComponentFrame.java

```
1  package v2ch11.text;
2
3  import module java.desktop;
4
5  /**
6   * A frame with sample text components.
7   */
```

```
8  public class TextComponentFrame extends JFrame {
9      private static final int TEXTAREA_ROWS = 8;
10     private static final int TEXTAREA_COLUMNS = 20;
11
12     public TextComponentFrame() {
13         var textField = new JTextField();
14         var passwordField = new JPasswordField();
15
16         var northPanel = new JPanel();
17         northPanel.setLayout(new GridLayout(2, 2));
18         northPanel.add(new JLabel("User name: ", SwingConstants.RIGHT));
19         northPanel.add(textField);
20         northPanel.add(new JLabel("Password: ", SwingConstants.RIGHT));
21         northPanel.add(passwordField);
22
23         add(northPanel, BorderLayout.NORTH);
24
25         var textArea = new JTextArea(TEXTAREA_ROWS, TEXTAREA_COLUMNS);
26         var scrollPane = new JScrollPane(textArea);
27
28         add(scrollPane, BorderLayout.CENTER);
29
30         // add button to append text into the text area
31
32         var southPanel = new JPanel();
33
34         var insertButton = new JButton("Insert");
35         southPanel.add(insertButton);
36         insertButton
37                 .addActionListener(_ -> textArea.append("User name: " + textField.getText()
38                         + " Password: " + new String(passwordField.getPassword()) + "\n"));
39
40         add(southPanel, BorderLayout.SOUTH);
41         pack();
42     }
43 }
```

`javax.swing.JTextArea` 1.2

- `JTextArea()`
- `JTextArea(int rows, int columns)`
- `JTextArea(String text, int rows, int columns)`
  construct a new text area.
- `void setColumns(int columns)`
  tells the text area the preferred number of columns it should use.
- `void setRows(int rows)`
  tells the text area the preferred number of rows it should use.
- `void append(String newText)`
  appends the given text to the end of the text already in the text area.
- `void setLineWrap(boolean wrap)`
  turns line wrapping on or off.
- `void setWrapStyleWord(boolean word)`
  If `word` is `true`, long lines are wrapped at word boundaries. If it is `false`, long lines are broken without taking word boundaries into account.

- `void setTabSize(int c)`
  sets tab stops every `c` columns. Note that the tabs aren't converted to spaces but cause alignment with the next tab stop.

`javax.swing.JScrollPane` 1.2

- `JScrollPane(Component c)`
  creates a scroll pane that displays the content of the specified component. Scrollbars are supplied when the component is larger than the view.

11.4. Choice Components

You now know how to collect text input from users, but there are many occasions where you would rather give users a finite set of choices than have them enter the data in a text component. Using a set of buttons or a list of items tells your users what choices they have. (It also saves you the trouble of error checking.) In this section, you will learn how to program checkboxes, radio buttons, lists of choices, and sliders.

11.4.1. Checkboxes

If you want to collect just a "yes" or "no" input, use a checkbox component. Checkboxes automatically come with labels that identify them. The user can check the box by clicking inside it and turn off the checkmark by clicking inside the box again. Pressing the space bar when the focus is in the checkbox also toggles the checkmark.

Figure 11.12 shows a simple program with two checkboxes, one for turning the italic attribute of a font on or off, and the other for boldface. Note that the second checkbox has focus, as indicated by the rectangle around the label. Each time the user clicks one of the checkboxes, the screen is refreshed, using the new font attributes.

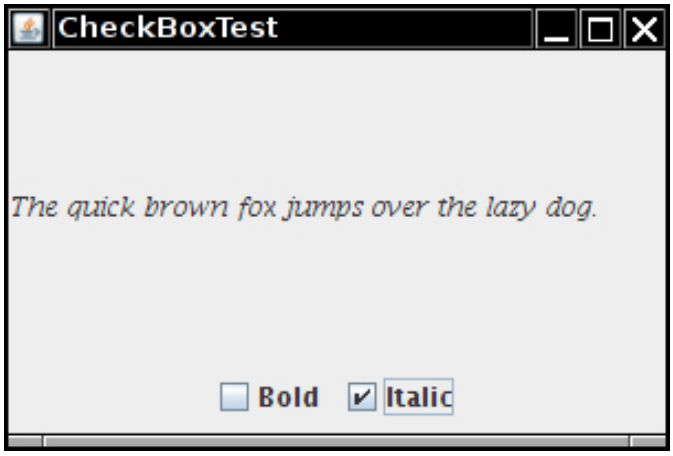


Figure 11.12: Checkboxes

Checkboxes need a label next to them to identify their purpose. Give the label text in the constructor:

```
bold = new JCheckBox("Bold");
```

Use the `setSelected` method to turn a checkbox on or off. For example:

```
bold.setSelected(true);
```

The `isSelected` method then retrieves the current state of each checkbox. It is `false` if unchecked, `true` if checked.

When the user clicks on a checkbox, this triggers an action event. As always, you attach an action listener to the checkbox. In our program, the two checkboxes share the same action listener.

```
ActionListener listener = . . .;
bold.addActionListener(listener);
italic.addActionListener(listener);
```

The listener queries the state of the `bold` and `italic` checkboxes and sets the font of the panel to plain, bold, italic, or both bold and italic.

```
ActionListener listener = _ -> {
    int mode = 0;
    if (bold.isSelected()) mode += Font.BOLD;
    if (italic.isSelected()) mode += Font.ITALIC;
    label.setFont(new Font(Font.SERIF, mode, FONTSIZE));
};
```

Listing 11.2 is the program listing for the checkbox example.

Listing 11.2 v2ch11/checkBox/CheckBoxFrame.java

```
1  package v2ch11.checkBox;
2
3  import module java.desktop;
4
5  /**
6   * A frame with a sample text label and check boxes for selecting font attributes.
7   */
8  public class CheckBoxFrame extends JFrame {
9      private JLabel label;
10     private JCheckBox bold;
11     private JCheckBox italic;
12     private static final int FONTSIZE = 24;
13
14     public CheckBoxFrame() {
15         // add the sample text label
16
17         label = new JLabel("The quick brown fox jumps over the lazy dog.");
18         label.setFont(new Font("Serif", Font.BOLD, FONTSIZE));
19         add(label, BorderLayout.CENTER);
20
21         // this listener sets the font attribute of
22         // the label to the check box state
23
24         ActionListener listener = _ -> {
25             int mode = 0;
26             if (bold.isSelected()) mode += Font.BOLD;
27             if (italic.isSelected()) mode += Font.ITALIC;
28             label.setFont(new Font("Serif", mode, FONTSIZE));
29         };
30
```

```
31          // add the check boxes
32          var buttonPanel = new JPanel();
33
34          bold = new JCheckBox("Bold");
35          bold.addActionListener(listener);
36          bold.setSelected(true);
37          buttonPanel.add(bold);
38
39          italic = new JCheckBox("Italic");
40          italic.addActionListener(listener);
41          buttonPanel.add(italic);
42
43          add(buttonPanel, BorderLayout.SOUTH);
44          pack();
45      }
46  }
```

`javax.swing.JCheckBox` 1.2

- `JCheckBox(String label)`
- `JCheckBox(String label, Icon icon)`

  construct a checkbox that is initially unselected.
- `JCheckBox(String label, boolean state)`

  constructs a checkbox with the given label and initial state.
- `boolean isSelected()`
- `void setSelected(boolean state)`

  get or set the selection state of the checkbox.

11.4.2. Radio Buttons

In the previous example, the user could check either, both, or neither of the two checkboxes. In many cases, we want the user to check only one of several boxes. When another box is checked, the previous box is automatically unchecked. Such a group of boxes is often called a *radio button group* because the buttons work like the station selector buttons on a radio. When you push in one button, the previously depressed button pops out.
Figure 11.13 shows a typical example. We allow the user to select a font size from among the choices—Small, Medium, Large, or Extra large—but, of course, we will allow selecting only one size at a time.



Figure 11.13: A radio button group

Implementing radio button groups is easy in Swing. You construct one object of type `ButtonGroup` for every group of buttons. Then, you add objects of type `JRadioButton` to the button group. The button group object is responsible for turning off the previously set button when a new button is clicked.

```
var group = new ButtonGroup();

var smallButton = new JRadioButton("Small", false);
group.add(smallButton);

var mediumButton = new JRadioButton("Medium", true);
group.add(mediumButton);
. . .
```

The second argument of the constructor is `true` for the button that should be checked initially and `false` for all others. Note that the button group controls only the *behavior* of the buttons; if you want to group the buttons for layout purposes, you also need to add them to a container such as a `JPanel`.

If you look again at **Figure 11.12**, you will note that the appearance of the radio buttons is different from that of checkboxes in **Figure 11.13**. Checkboxes are square and contain a checkmark when selected. Radio buttons are round and contain a dot when selected.

The event notification mechanism for radio buttons is the same as for any other buttons. When the user checks a radio button, the button generates an action event. In our example program, we define an action listener that sets the font size to a particular value:

```
ActionListener listener = _ -> label.setFont(new Font("Serif", Font.PLAIN, size));
```

Compare this listener setup to that of the checkbox example. Each radio button gets a different listener object. Each listener object knows exactly what it needs to do—set the font size to a particular value. With checkboxes, we used a different approach: Both checkboxes have the same action listener that calls a method looking at the current state of both checkboxes.

Could we follow the same approach here? We could have a single listener that computes the size as follows:

```
if (smallButton.isSelected()) size = 8;
else if (mediumButton.isSelected()) size = 12;
. . .
```

However, we prefer to use separate action listener objects because they tie the size values more closely to the buttons.



Note: If you have a group of radio buttons, you know that only one of them is selected. It would be nice to be able to quickly find out which, without having to query all the buttons in the group. The `ButtonGroup` object controls all buttons, so it would be convenient if this object could give us a reference to the selected button. Indeed, the `ButtonGroup` class has a `getSelection` method, but that method doesn't return the radio button that is selected. Instead, it returns a `ButtonModel` reference to the model attached to the button. Unfortunately, none of the `ButtonModel` methods are very helpful. The `ButtonModel` interface inherits a method `getSelectedObjects` from the `ItemSelectable` interface that, rather uselessly, returns `null`. The `getActionCommand` method looks promising because the "action command" of a radio button is its text label. But the action command of its model is `null`. Only if you explicitly set the action

commands of all radio buttons with the `setActionCommand` method do the action command values of the models also get set. Then you can retrieve the action command of the currently selected button with `buttonGroup.getSelection().getActionCommand()`.

Listing 11.3 is the complete program for font size selection that puts a set of radio buttons to work.

Listing 11.3 v2ch11/radioButton/RadioButtonFrame.java

```
1  package v2ch11.radioButton;
2
3  import module java.desktop;
4
5  /**
6   * A frame with a sample text label and radio buttons for selecting font sizes.
7   */
8  public class RadioButtonFrame extends JFrame {
9      private JPanel buttonPanel;
10     private ButtonGroup group;
11     private JLabel label;
12     private static final int DEFAULT_SIZE = 36;
13
14     public RadioButtonFrame() {
15         // add the sample text label
16
17         label = new JLabel("The quick brown fox jumps over the lazy dog.");
18         label.setFont(new Font("Serif", Font.PLAIN, DEFAULT_SIZE));
19         add(label, BorderLayout.CENTER);
20
21         // add the radio buttons
22
23         buttonPanel = new JPanel();
24         group = new ButtonGroup();
25
26         addRadioButton("Small", 8);
27         addRadioButton("Medium", 12);
28         addRadioButton("Large", 18);
29         addRadioButton("Extra large", 36);
30
31         add(buttonPanel, BorderLayout.SOUTH);
32         pack();
33     }
34
35     /**
36      * Adds a radio button that sets the font size of the sample text.
37      * @param name the string to appear on the button
38      * @param size the font size that this button sets
39      */
40     public void addRadioButton(String name, int size) {
41         boolean selected = size == DEFAULT_SIZE;
42         var button = new JRadioButton(name, selected);
43         group.add(button);
44         buttonPanel.add(button);
45
46         // this listener sets the label font size
47
48         ActionListener listener = _ -> label.setFont(new Font("Serif", Font.PLAIN, size));
49
```

```
50          button.addActionListener(listener);
51      }
52  }
```

`javax.swing.JRadioButton` 1.2

- `JRadioButton(String label, Icon icon)`
  constructs a radio button that is initially unselected.
- `JRadioButton(String label, boolean state)`
  constructs a radio button with the given label and initial state.

`javax.swing.ButtonGroup` 1.2

- `void add(AbstractButton b)`
  adds the button to the group.
- `ButtonModel getSelection()`
  returns the button model of the selected button.

`javax.swing.ButtonModel` 1.2

- `String getActionCommand()`
  returns the action command for this button model.

`javax.swing.AbstractButton` 1.2

- `void setActionCommand(String s)`
  sets the action command for this button and its model.

11.4.3. Borders

If you have multiple groups of radio buttons in a window, you will want to visually indicate which buttons are grouped. Swing provides a set of useful *borders* for this purpose. You can apply a border to any component that extends `JComponent`. The most common usage is to place a border around a panel and fill that panel with other user interface elements, such as radio buttons.

You can choose from quite a few borders, but you need to follow the same steps for all of them.

1. Call a static method of the `BorderFactory` to create a border. You can choose among the following styles (see **Figure 11.14**):
   1. Lowered bevel
   2. Raised bevel
   3. Etched
   4. Line
   5. Matte
   6. Empty (just to create some blank space around the component)

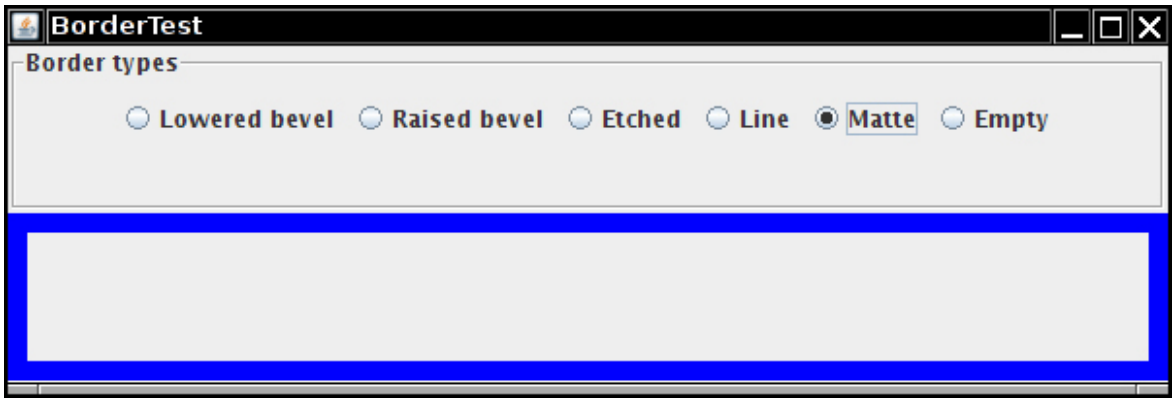


Figure 11.14: Testing border types

2. If you like, add a title to your border by passing your border to `BorderFactory.createTitledBorder`.
3. If you really want to go all out, combine several borders with a call to `BorderFactory.createCompoundBorder`.
4. Add the resulting border to your component by calling the `setBorder` method of the `JComponent` class.

For example, here is how you add an etched border with a title to a panel:

```
Border etched = BorderFactory.createEtchedBorder();
Border titled = BorderFactory.createTitledBorder(etched, "A Title");
panel.setBorder(titled);
```

Different borders have different options for setting border widths and colors; see the API notes for details. True border enthusiasts will appreciate that there is also a `SoftBevelBorder` class for beveled borders with softened corners and that a `LineBorder` can have rounded corners as well. You can construct these borders only by using one of the class constructors—there is no `BorderFactory` method for them.

`javax.swing.BorderFactory` 1.2

- `static Border createLineBorder(Color color)`
- `static Border createLineBorder(Color color, int thickness)`
  create a simple line border.
- `static MatteBorder createMatteBorder(int top, int left, int bottom, int right, Color color)`
- `static MatteBorder createMatteBorder(int top, int left, int bottom, int right, Icon tileIcon)`
  create a thick border that is filled with a color or a repeating icon.
- `static Border createEmptyBorder()`
- `static Border createEmptyBorder(int top, int left, int bottom, int right)`
  create an empty border.
- `static Border createEtchedBorder()`
- `static Border createEtchedBorder(Color highlight, Color shadow)`
- `static Border createEtchedBorder(int type)`
- `static Border createEtchedBorder(int type, Color highlight, Color shadow)`
  create a line border with a 3D effect. The `type` parameter is one of `EtchedBorder.RAISED`, `EtchedBorder.LOWERED`.

- `static Border createBevelBorder(int type)`
- `static Border createBevelBorder(int type, Color highlight, Color shadow)`
- `static Border createLoweredBevelBorder()`
- `static Border createRaisedBevelBorder()`
  create a border that gives the effect of a lowered or raised surface. The `type` parameter is one of `BevelBorder.RAISED`, `BevelBorder.LOWERED`.
- `static TitledBorder createTitledBorder(String title)`
- `static TitledBorder createTitledBorder(Border border)`
- `static TitledBorder createTitledBorder(Border border, String title)`
- `static TitledBorder createTitledBorder(Border border, String title, int justification, int position)`
- `static TitledBorder createTitledBorder(Border border, String title, int justification, int position, Font font)`
- `static TitledBorder createTitledBorder(Border border, String title, int justification, int position, Font font, Color color)`
  create a titled border with the specified properties. The `justification` parameter is one of the `TitledBorder` constants `LEFT`, `CENTER`, `RIGHT`, `LEADING`, `TRAILING`, or `DEFAULT_JUSTIFICATION` (left), and `position` is one of `ABOVE_TOP`, `TOP`, `BELOW_TOP`, `ABOVE_BOTTOM`, `BOTTOM`, `BELOW_BOTTOM`, or `DEFAULT_POSITION` (top).
- `static CompoundBorder createCompoundBorder(Border outsideBorder, Border insideBorder)`
  combines two borders into a new border.

`javax.swing.border.SoftBevelBorder` 1.2

- `SoftBevelBorder(int type)`
- `SoftBevelBorder(int type, Color highlight, Color shadow)`
  create a bevel border with softened corners. The `type` parameter is one of `SoftBevelBorder.RAISED`, `SoftBevelBorder.LOWERED`.

`javax.swing.border.LineBorder` 1.2

- `public LineBorder(Color color, int thickness, boolean roundedCorners)`
  creates a line border with the given color and thickness. If `roundedCorners` is `true`, the border has rounded corners.

`javax.swing.JComponent` 1.2

- `void setBorder(Border border)`
  sets the border of this component.

11.4.4. Combo Boxes

If you have more than a handful of alternatives, radio buttons are not a good choice because they take up too much screen space. Instead, you can use a combo box. When the user clicks on this component, a list of choices drops down, and the user can then select one of them (see **Figure 11.15**).

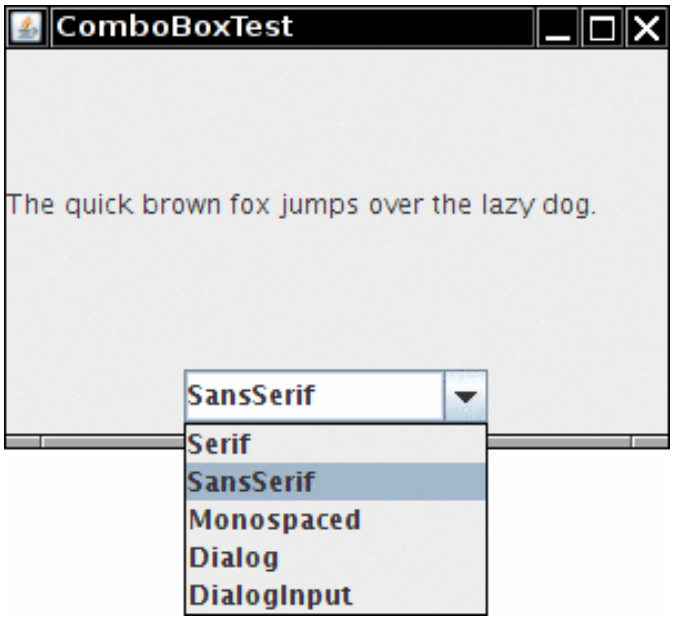


Figure 11.15: A combo box

If the drop-down list box is set to be *editable*, you can edit the current selection as if it were a text field. For that reason, this component is called a *combo box*—it combines the flexibility of a text field with a set of predefined choices. The `JComboBox` class provides a combo box component.

As of Java 7, the `JComboBox` class is a generic class. For example, a `JComboBox<String>` holds objects of type `String`, and a `JComboBox<Integer>` holds integers.

Call the `setEditable` method to make the combo box editable. Note that editing affects only the selected item. It does not change the list of choices in any way.

You can obtain the current selection, which may have been edited if the combo box is editable, by calling the `getSelectedItem` method. However, for an editable combo box, that item may have any type, depending on the editor that takes the user edits and turns the result into an object. (See **Chapter 12** for a discussion of editors.) If your combo box isn't editable, you are better off calling

```
combo.getItemAt(combo.getSelectedIndex())
```

which gives you the selected item with the correct type.

In the example program, the user can choose a font style from a list of styles (Serif, SansSerif, Monospaced, etc.). The user can also type in another font.

Add the choice items with the `addItem` method. In our program, `addItem` is called only in the constructor, but you can call it any time.

```
var faceCombo = new JComboBox<String>();
faceCombo.addItem("Serif");
faceCombo.addItem("SansSerif");
. . .
```

This method adds the string to the end of the list. You can add new items anywhere in the list with the `insertItemAt` method:

```
faceCombo.insertItemAt("Monospaced", 0); // add at the beginning
```

You can add items of any type—the combo box invokes each item's `toString` method to display it.

If you need to remove items at runtime, use the `removeItem` or `removeItemAt` method, depending on whether you supply the item to be removed or its position.

```
faceCombo.removeItem("Monospaced");
faceCombo.removeItemAt(0); // remove first item
```

The `removeAllItems` method removes all items at once.



Tip: If you need to add a large number of items to a combo box, the `addItem` method will perform poorly. Instead, construct a `DefaultComboBoxModel`, populate it by calling `addElement`, and then call the `setModel` method of the `JComboBox` class.

When the user selects an item from a combo box, the combo box generates an action event. To find out which item was selected, call `getSource` on the event parameter to get a reference to the combo box that sent the event. Then call the `getSelectedItem` method to retrieve the currently selected item. You will need to cast the returned value to the appropriate type, usually `String`.

```
ActionListener listener = _ ->
    label.setFont(new Font(
        faceCombo.getItemAt(faceCombo.getSelectedIndex()),
        Font.PLAIN,
        DEFAULT_SIZE));
```

Listing 11.4 shows the complete program.

Listing 11.4 v2ch11/comboBox/ComboBoxFrame.java

```
1  package v2ch11.comboBox;
2
3  import module java.desktop;
4
5  /**
6   * A frame with a sample text label and a combo box for selecting font faces.
7   */
8  public class ComboBoxFrame extends JFrame {
9      private JComboBox<String> faceCombo;
10     private JLabel label;
11     private static final int DEFAULT_SIZE = 24;
12
13     public ComboBoxFrame() {
14         // add the sample text label
15
16         label = new JLabel("The quick brown fox jumps over the lazy dog.");
17         label.setFont(new Font("Serif", Font.PLAIN, DEFAULT_SIZE));
18         add(label, BorderLayout.CENTER);
19
20         // make a combo box and add face names
21
```

```
        faceCombo = new JComboBox<>();
        faceCombo.addItem("Serif");
        faceCombo.addItem("SansSerif");
        faceCombo.addItem("Monospaced");
        faceCombo.addItem("Dialog");
        faceCombo.addItem("DialogInput");

        // the combo box listener changes the label font to the selected face name

        faceCombo.addActionListener(_ -> label.setFont(new Font(
                faceCombo.getItemAt(faceCombo.getSelectedIndex()), Font.PLAIN, DEFAULT_SIZE)));

        // add combo box to a panel at the frame's southern border

        var comboPanel = new JPanel();
        comboPanel.add(faceCombo);
        add(comboPanel, BorderLayout.SOUTH);
        pack();
    }
}
```

`javax.swing.JComboBox` 1.2

- `boolean isEditable()`
- `void setEditable(boolean b)`
  get or set the `editable` property of this combo box.
- `void addItem(Object item)`
  adds an item to the item list.
- `void insertItemAt(Object item, int index)`
  inserts an item into the item list at a given index.
- `void removeItem(Object item)`
  removes an item from the item list.
- `void removeItemAt(int index)`
  removes the item at an index.
- `void removeAllItems()`
  removes all items from the item list.
- `Object getSelectedItem()`
  returns the currently selected item.

11.4.5. Sliders

Combo boxes let users choose from a discrete set of values. Sliders offer a choice from a continuum of values—for example, any number between 1 and 100.

The most common way of constructing a slider is as follows:

```
var slider = new JSlider(min, max, initialValue);
```

If you omit the minimum, maximum, and initial values, they are initialized with `0`, `100`, and `50`, respectively.

Or if you want the slider to be vertical, use the following constructor call:

```
var slider = new JSlider(SwingConstants.VERTICAL, min, max, initialValue);
```

These constructors create a plain slider, such as the top slider in **Figure 11.16**. You will see presently how to add decorations to a slider.

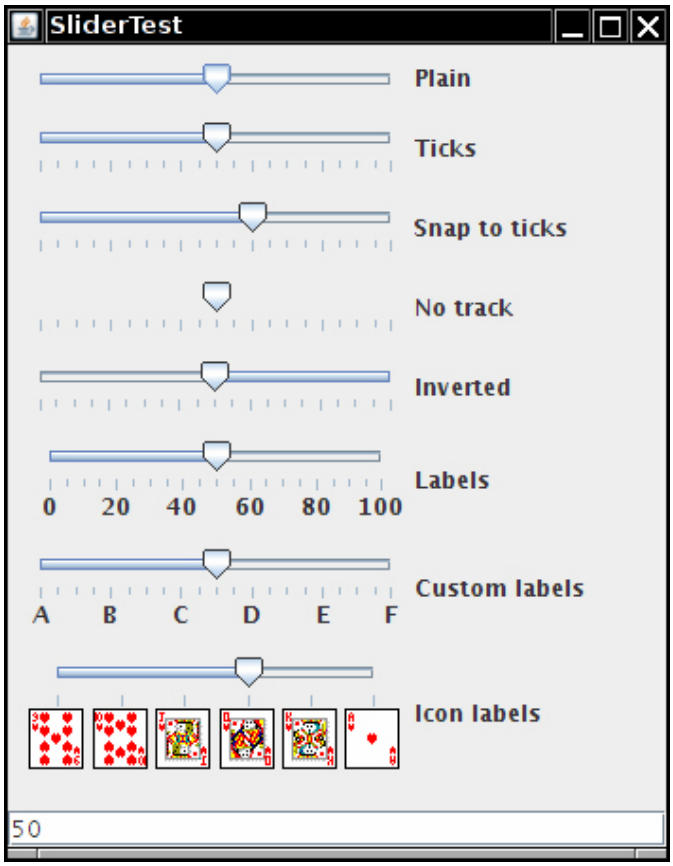


Figure 11.16: Sliders

As the user slides the slider bar, the *value* of the slider moves between the minimum and the maximum values. When the value changes, a `ChangeEvent` is sent to all change listeners. To be notified of the change, call the `addChangeListener` method and install an object that implements the functional `ChangeListener` interface. In the callback, retrieve the slider value:

```
ChangeListener listener = _ -> {
    JSlider slider = (JSlider) event.getSource();
    int value = slider.getValue();
    . . .
};
```

You can embellish the slider by showing *ticks*. For example, in the sample program, the second slider uses the following settings:

```
slider.setMajorTickSpacing(20);
slider.setMinorTickSpacing(5);
```

The slider is decorated with large tick marks every 20 units and small tick marks every 5 units. The units refer to slider values, not pixels.

These instructions only set the units for the tick marks. To actually have the tick marks appear, call

```
slider.setPaintTicks(true);
```

The major and minor tick marks are independent. For example, you can set major tick marks every 20 units and minor tick marks every 7 units, but that will give you a very messy scale.

You can force the slider to *snap to ticks*. Whenever the user has finished dragging a slider in snap mode, it is immediately moved to the closest tick. You activate this mode with the call

```
slider.setSnapToTicks(true);
```



Caution: The "snap to ticks" behavior doesn't work as well as you might imagine. Until the slider has actually snapped, the change listener still reports slider values that don't correspond to ticks. And if you click next to the slider—an action that normally advances the slider a bit in the direction of the click—a slider with "snap to ticks" does not move to the next tick.

You can display *tick mark labels* for the major tick marks by calling

```
slider.setPaintLabels(true);
```

For example, with a slider ranging from 0 to 100 and major tick spacing of 20, the ticks are labeled 0, 20, 40, 60, 80, and 100.

You can also supply other tick mark labels, such as strings or icons (see **Figure 11.16**). The process is a bit convoluted. You need to fill a hash table with keys of type `Integer` and values of type `Component`. You then call the `setLabelTable` method. The components are placed under the tick marks. Usually, `JLabel` objects are used. Here is how you can label ticks as A, B, C, D, E, and F:

```
var labelTable = new Hashtable<Integer, Component>();
labelTable.put(0, new JLabel("A"));
labelTable.put(20, new JLabel("B"));
. . .
labelTable.put(100, new JLabel("F"));
slider.setLabelTable(labelTable);
```

Listing 11.5 also shows a slider with icons as tick labels.



Tip: If your tick marks or labels don't show, double-check that you called `setPaintTicks(true)` and `setPaintLabels(true)`.

The fourth slider in **Figure 11.16** has no track. To suppress the "track" in which the slider moves, call

```
slider.setPaintTrack(false);
```

The fifth slider has its direction reversed by a call to

```
slider.setInverted(true);
```

The example program in **Listing 11.5** shows all these visual effects with a collection of sliders. Each slider has a change event listener installed that places the current slider value into the text field at the bottom of the frame.

Listing 11.5 v2ch11/slider/SliderFrame.java

```
package v2ch11.slider;

import module java.base;
import module java.desktop;

/**
 * A frame with many sliders and a text field to show slider values.
 */
public class SliderFrame extends JFrame {
    private JPanel sliderPanel;
    private JTextField textField;
    private ChangeListener listener;

    public SliderFrame() {
        sliderPanel = new JPanel();
        sliderPanel.setLayout(new GridBagLayout());

        // common listener for all sliders
        listener = event -> {
            // update text field when the slider value changes
            JSlider source = (JSlider) event.getSource();
            textField.setText("" + source.getValue());
        };

        // add a plain slider

        var slider = new JSlider();
        addSlider(slider, "Plain");

        // add a slider with major and minor ticks

        slider = new JSlider();
        slider.setPaintTicks(true);
        slider.setMajorTickSpacing(20);
        slider.setMinorTickSpacing(5);
        addSlider(slider, "Ticks");

        // add a slider that snaps to ticks

        slider = new JSlider();
        slider.setPaintTicks(true);
        slider.setSnapToTicks(true);
        slider.setMajorTickSpacing(20);
        slider.setMinorTickSpacing(5);
        addSlider(slider, "Snap to ticks");

        // add a slider with no track

        slider = new JSlider();
        slider.setPaintTicks(true);
        slider.setMajorTickSpacing(20);
        slider.setMinorTickSpacing(5);
```

```
        slider.setPaintTrack(false);
        addSlider(slider, "No track");

        // add an inverted slider

        slider = new JSlider();
        slider.setPaintTicks(true);
        slider.setMajorTickSpacing(20);
        slider.setMinorTickSpacing(5);
        slider.setInverted(true);
        addSlider(slider, "Inverted");

        // add a slider with numeric labels

        slider = new JSlider();
        slider.setPaintTicks(true);
        slider.setPaintLabels(true);
        slider.setMajorTickSpacing(20);
        slider.setMinorTickSpacing(5);
        addSlider(slider, "Labels");

        // add a slider with alphabetic labels

        slider = new JSlider();
        slider.setPaintLabels(true);
        slider.setPaintTicks(true);
        slider.setMajorTickSpacing(20);
        slider.setMinorTickSpacing(5);

        var labelTable = new Hashtable<Integer, Component>();
        labelTable.put(0, new JLabel("A"));
        labelTable.put(20, new JLabel("B"));
        labelTable.put(40, new JLabel("C"));
        labelTable.put(60, new JLabel("D"));
        labelTable.put(80, new JLabel("E"));
        labelTable.put(100, new JLabel("F"));

        slider.setLabelTable(labelTable);
        addSlider(slider, "Custom labels");

        // add a slider with icon labels

        slider = new JSlider();
        slider.setPaintTicks(true);
        slider.setPaintLabels(true);
        slider.setSnapToTicks(true);
        slider.setMajorTickSpacing(20);
        slider.setMinorTickSpacing(20);

        labelTable = new Hashtable<>();

        // add card images

        labelTable.put(0, new JLabel(new ImageIcon("images/nine.gif")));
        labelTable.put(20, new JLabel(new ImageIcon("images/ten.gif")));
        labelTable.put(40, new JLabel(new ImageIcon("images/jack.gif")));
        labelTable.put(60, new JLabel(new ImageIcon("images/queen.gif")));
        labelTable.put(80, new JLabel(new ImageIcon("images/king.gif")));
```

```
        labelTable.put(100, new JLabel(new ImageIcon("images/ace.gif")));

        slider.setLabelTable(labelTable);
        addSlider(slider, "Icon labels");

        // add the text field that displays the slider value

        textField = new JTextField();
        add(sliderPanel, BorderLayout.CENTER);
        add(textField, BorderLayout.SOUTH);
        pack();
    }

    /**
     * Adds a slider to the slider panel and hooks up the listener.
     * @param slider the slider
     * @param description the slider description
     */
    public void addSlider(JSlider slider, String description) {
        slider.addChangeListener(listener);
        var panel = new JPanel();
        panel.add(slider);
        panel.add(new JLabel(description));
        panel.setAlignmentX(Component.LEFT_ALIGNMENT);
        var gbc = new GridBagConstraints();
        gbc.gridy = sliderPanel.getComponentCount();
        gbc.anchor = GridBagConstraints.WEST;
        sliderPanel.add(panel, gbc);
    }
}
```

`javax.swing.JSlider` 1.2

- `JSlider()`
- `JSlider(int direction)`
- `JSlider(int min, int max)`
- `JSlider(int min, int max, int initialValue)`
- `JSlider(int direction, int min, int max, int initialValue)`
  construct a slider with the given direction and minimum, maximum, and initial values. The `direction` parameter is one of `SwingConstants.HORIZONTAL` or `SwingConstants.VERTICAL`. The default is horizontal. Defaults for the minimum, initial, and maximum are `0`, `50`, and `100`.
- `void setPaintTicks(boolean b)`
  displays ticks if `b` is `true`.
- `void setMajorTickSpacing(int units)`
- `void setMinorTickSpacing(int units)`
  set major or minor ticks at multiples of the given slider units.
- `void setPaintLabels(boolean b)`
  displays tick labels if `b` is `true`.
- `void setLabelTable(Dictionary table)`
  sets the components to use for the tick labels. Each key/value pair in the table has the form `Integer.valueOf(`*`value`*`)/`*`component`*.
- `void setSnapToTicks(boolean b)`
  If `b` is `true`, then the slider snaps to the closest tick after each adjustment.

- `void setPaintTrack(boolean b)`
  If `b` is `true`, a track is displayed in which the slider runs.

11.5. Menus

This chapter started by introducing the most common components that you might want to place into a window, such as various kinds of buttons, text fields, and combo boxes. Swing also supports another type of user interface element—pull-down menus that are familiar from GUI applications.

A *menu bar* at the top of a window contains the names of the pull-down menus. Clicking on a name opens the menu containing *menu items* and *submenus*. When the user clicks on a menu item, all menus are closed and a message is sent to the program. **Figure 11.17** shows a typical menu with a submenu.

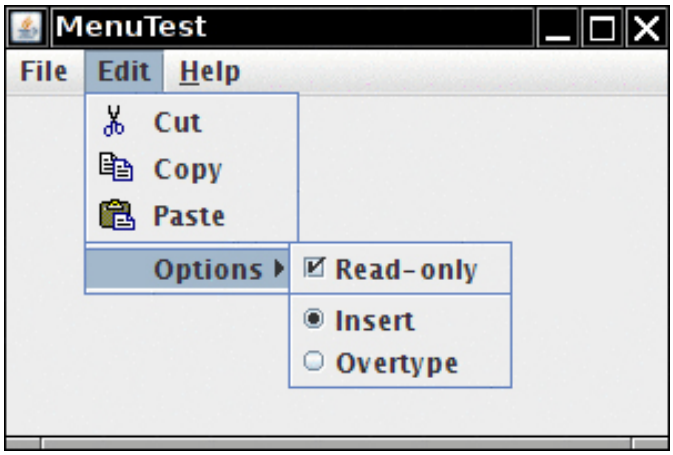


Figure 11.17: A menu with a submenu

11.5.1. Menu Building

Building menus is straightforward. First, create a menu bar:

```
var menuBar = new JMenuBar();
```

A menu bar is just a component that you can add anywhere you like. Normally, you want it to appear at the top of a frame. You can add it there with the `setJMenuBar` method:

```
frame.setJMenuBar(menuBar);
```

For each menu, you create a menu object:

```
var editMenu = new JMenu("Edit");
```

Add the top-level menus to the menu bar:

```
menuBar.add(editMenu);
```

Add menu items, separators, and submenus to the menu object:

```
var pasteItem = new JMenuItem("Paste");
editMenu.add(pasteItem);
editMenu.addSeparator();
JMenu optionsMenu = . . .; // a submenu
editMenu.add(optionsMenu);
```

You can see separators in **Figure 11.17** below the Paste and Read-only menu items.

When the user selects a menu item, an action event is triggered. You need to install an action listener for each menu item:

```
ActionListener listener = . . .;
pasteItem.addActionListener(listener);
```

The method `JMenu.add(String s)` conveniently adds a menu item to the end of a menu. For example:

```
editMenu.add("Paste");
```

The `add` method returns the created menu item, so you can capture it and add the listener, as follows:

```
JMenuItem pasteItem = editMenu.add("Paste");
pasteItem.addActionListener(listener);
```

It often happens that menu items trigger commands that can also be activated through other user interface elements such as toolbar buttons. In **Chapter 10**, you saw how to specify commands through `Action` objects. You define a class that implements the `Action` interface, usually by extending the `AbstractAction` convenience class, specify the menu item label in the constructor of the `AbstractAction` object, and override the `actionPerformed` method to hold the menu action handler. For example:

```
var exitAction = new AbstractAction("Exit") { // menu item text goes here
    public void actionPerformed(ActionEvent event) {
        // action code goes here
        System.exit(0);
    }
};
```

You can then add the action to the menu:

```
JMenuItem exitItem = fileMenu.add(exitAction);
```

This command adds a menu item to the menu, using the action name. The action object becomes its listener. This is just a convenient shortcut for

```
var exitItem = new JMenuItem(exitAction);
fileMenu.add(exitItem);
```

`javax.swing.JMenu` 1.2

- `JMenu(String label)`
  constructs a menu with the given label.
- `JMenuItem add(JMenuItem item)`
  adds a menu item (or a menu).
- `JMenuItem add(String label)`
  adds a menu item with the given label to this menu and returns the item.
- `JMenuItem add(Action a)`
  adds a menu item with the given action to this menu and returns the item.
- `void addSeparator()`
  adds a separator line to the menu.
- `JMenuItem insert(JMenuItem menu, int index)`
  adds a new menu item (or submenu) to the menu at a specific index.
- `JMenuItem insert(Action a, int index)`
  adds a new menu item with the given action at a specific index.
- `void insertSeparator(int index)`
  adds a separator to the menu.
- `void remove(int index)`
- `void remove(JMenuItem item)`
  remove a specific item from the menu.

`javax.swing.JMenuItem` 1.2

- `JMenuItem(String label)`
  constructs a menu item with a given label.
- `JMenuItem(Action a)` 1.3
  constructs a menu item for the given action.

`javax.swing.AbstractButton` 1.2

- `void setAction(Action a)` 1.3
  sets the action for this button or menu item.

`javax.swing.JFrame` 1.2

- `void setJMenuBar(JMenuBar menubar)`
  sets the menu bar for this frame.

11.5.2. Icons in Menu Items

Menu items are very similar to buttons. In fact, the `JMenuItem` class extends the `AbstractButton` class. Just like buttons, menus can have just a text label, just an icon, or both. You can specify the icon with the `JMenuItem(String, Icon)` or `JMenuItem(Icon)` constructor, or you can set it with the `setIcon` method that the `JMenuItem` class inherits from the `AbstractButton` class. Here is an example:

```
var cutItem = new JMenuItem("Cut", new ImageIcon("cut.gif"));
```

In Figure 11.17, you can see icons next to several menu items. By default, the menu item text is placed to the right of the icon. If you prefer the text to be placed on the left, call the `setHorizontalTextPosition` method that the `JMenuItem` class inherits from the `AbstractButton` class. For example, the call

```
cutItem.setHorizontalTextPosition(SwingConstants.LEFT);
```

moves the menu item text to the left of the icon.

You can also add an icon to an action:

```
cutAction.putValue(Action.SMALL_ICON, new ImageIcon("cut.gif"));
```

Whenever you construct a menu item out of an action, the `Action.NAME` value becomes the text of the menu item and the `Action.SMALL_ICON` value becomes the icon.

Alternatively, you can set the icon in the `AbstractAction` constructor:

```
cutAction = new AbstractAction("Cut", new ImageIcon("cut.gif")) {
        public void actionPerformed(ActionEvent event) {
            . . .
        }
    };
```

`javax.swing.JMenuItem` 1.2

- `JMenuItem(String label, Icon icon)`
  constructs a menu item with the given label and icon.

`javax.swing.AbstractButton` 1.2

- `void setHorizontalTextPosition(int pos)`
  sets the horizontal position of the text relative to the icon. The `pos` parameter is `SwingConstants.RIGHT` (text is to the right of icon) or `SwingConstants.LEFT`.

`javax.swing.AbstractAction` 1.2

- `AbstractAction(String name, Icon smallIcon)`
  constructs an abstract action with the given name and icon.

11.5.3. Checkbox and Radio Button Menu Items

Checkbox and *radio button* menu items display a checkbox or radio button next to the name (see Figure 11.17). When the user selects the menu item, the item automatically toggles between checked and unchecked.

Apart from the button decoration, treat these menu items just as you would any others. For example, here is how you create a checkbox menu item:

```
var readonlyItem = new JCheckBoxMenuItem("Read-only");
optionsMenu.add(readonlyItem);
```

The radio button menu items work just like regular radio buttons. You must add them to a button group. When one of the buttons in a group is selected, all others are automatically deselected.

```
var group = new ButtonGroup();
var insertItem = new JRadioButtonMenuItem("Insert");
insertItem.setSelected(true);
var overtypeItem = new JRadioButtonMenuItem("Overtype");
group.add(insertItem);
group.add(overtypeItem);
optionsMenu.add(insertItem);
optionsMenu.add(overtypeItem);
```

With these menu items, you don't necessarily want to be notified when the user selects the item. Instead, you can simply use the `isSelected` method to test the current state of the menu item. (Of course, that means you should keep a reference to the menu item stored in an instance field.) Use the `setSelected` method to set the state.

`javax.swing.JCheckBoxMenuItem` 1.2

- `JCheckBoxMenuItem(String label)`
  constructs the checkbox menu item with the given label.
- `JCheckBoxMenuItem(String label, boolean state)`
  constructs the checkbox menu item with the given label and the given initial state (`true` is checked).

`javax.swing.JRadioButtonMenuItem` 1.2

- `JRadioButtonMenuItem(String label)`
  constructs the radio button menu item with the given label.
- `JRadioButtonMenuItem(String label, boolean state)`
  constructs the radio button menu item with the given label and the given initial state (`true` is checked).

`javax.swing.AbstractButton` 1.2

- `boolean isSelected()`
- `void setSelected(boolean state)`
  get or set the selection state of this item (`true` is checked).

11.5.4. Pop-Up Menus

A *pop-up menu* is a menu that is not attached to a menu bar but floats somewhere (see **Figure 11.18**).

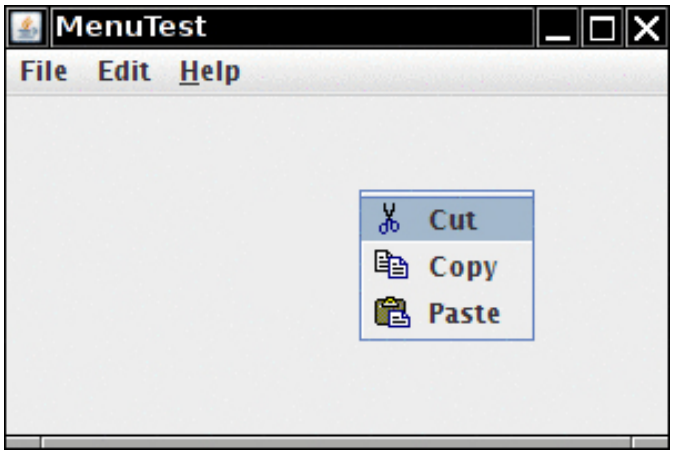


Figure 11.18: A pop-up menu

Create a pop-up menu just as you create a regular menu, except that a pop-up menu has no title.

```
var popup = new JPopupMenu();
```

Then, add your menu items as usual:

```
var item = new JMenuItem("Cut");
item.addActionListener(listener);
popup.add(item);
```

Unlike the regular menu bar that is always shown at the top of the frame, you must explicitly display a pop-up menu by using the `show` method. Specify the parent component and the location of the pop-up, using the coordinate system of the parent. For example:

```
popup.show(panel, x, y);
```

Usually, you want to pop up a menu when the user clicks a particular mouse button—the so-called *pop-up trigger*. In Windows and Linux, the pop-up trigger is the nonprimary (usually, the right) mouse button. To pop up a menu when the user clicks on a component, using the pop-up trigger, simply call the method

```
component.setComponentPopupMenu(popup);
```

Very occasionally, you may place a component inside another component that has a pop-up menu. The child component can inherit the parent component's pop-up menu by calling

```
child.setInheritsPopupMenu(true);
```

`javax.swing.JPopupMenu` 1.2

- `void show(Component c, int x, int y)`
  shows the pop-up menu over the component `c` with the top left corner at `(x, y)` (in the coordinate space of `c`).
- `boolean isPopupTrigger(MouseEvent event)` 1.3
  returns `true` if the mouse event is the pop-up menu trigger.

`java.awt.event.MouseEvent` 1.1

- `boolean isPopupTrigger()`
  returns `true` if this mouse event is the pop-up menu trigger.

`javax.swing.JComponent` 1.2

- `JPopupMenu getComponentPopupMenu()` 5.0
- `void setComponentPopupMenu(JPopupMenu popup)` 5.0
  get or set the pop-up menu for this component.
- `boolean getInheritsPopupMenu()` 5.0
- `void setInheritsPopupMenu(boolean b)` 5.0
  get or set the `inheritsPopupMenu` property. If the property is set and this component's pop-up menu is `null`, it uses its parent's pop-up menu.

11.5.5. Keyboard Mnemonics and Accelerators

It is a real convenience for the experienced user to select menu items by *keyboard mnemonics*. You can create a keyboard mnemonic for a menu item by specifying a mnemonic letter in the menu item constructor:

```
var aboutItem = new JMenuItem("About", 'A');
```

The keyboard mnemonic is displayed automatically in the menu, with the mnemonic letter underlined (see Figure 11.19). For example, in the item defined in the last example, the label will be displayed as "About" with an underlined letter 'A'. When the menu is displayed, the user just needs to press the A key, and the menu item is selected. (If the mnemonic letter is not part of the menu string, then typing it still selects the item, but the mnemonic is not displayed in the menu. Naturally, such invisible mnemonics are of dubious utility.)



Figure 11.19: Keyboard mnemonics

Sometimes, you don't want to underline the first letter of the menu item that matches the mnemonic. For example, if you have a mnemonic "A" for the menu item "Save As," then it makes more sense to underline the second "A" (Save As). You can specify which character you want to have underlined by calling the `setDisplayedMnemonicIndex` method.

If you have an `Action` object, you can add the mnemonic as the value of the `Action.MNEMONIC_KEY` key, as follows:

```
aboutAction.putValue(Action.MNEMONIC_KEY, Integer.valueOf('A'));
```

You can supply a mnemonic letter only in the constructor of a menu item, not in the constructor for a menu. To attach a mnemonic to a menu, call the `setMnemonic` method:

```
var helpMenu = new JMenu("Help");
helpMenu.setMnemonic('H');
```

To select a top-level menu from the menu bar, press the Alt key together with the mnemonic letter. For example, press Alt+H to select the Help menu from the menu bar.

Keyboard mnemonics let you select a submenu or menu item from the currently open menu. In contrast, *accelerators* are keyboard shortcuts that let you select menu items without ever opening a menu. For example, many programs attach the accelerators Ctrl+O and Ctrl+S to the Open and Save items in the File menu. Use the `setAccelerator` method to attach an accelerator key to a menu item. The `setAccelerator` method takes an object of type `Keystroke`. For example, the following call attaches the accelerator Ctrl+O to the `openItem` menu item:

```
openItem.setAccelerator(KeyStroke.getKeyStroke("ctrl O"));
```

Typing the accelerator key combination automatically selects the menu option and fires an action event, as if the user had selected the menu option manually.

You can attach accelerators only to menu items, not to menus. Accelerator keys don't actually open the menu. Instead, they directly fire the action event associated with a menu.

Conceptually, adding an accelerator to a menu item is similar to the technique of adding an accelerator to a Swing component. However, when the accelerator is added to a menu item, the key combination is automatically displayed in the menu (see **Figure 11.20**).

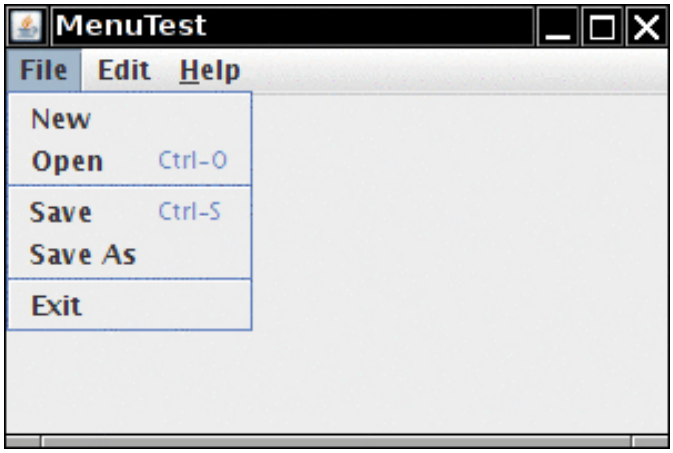


Figure 11.20: Accelerators



Note: Under Windows, Alt+F4 closes a window. But this is not an accelerator to be programmed in Java. It is a shortcut defined by the operating system. This key combination will always trigger the `WindowClosing` event for the active window regardless of whether there is a Close item on the menu.

`javax.swing.JMenuItem` 1.2

- `JMenuItem(String label, int mnemonic)`
  constructs a menu item with a given label and mnemonic.
- `void setAccelerator(KeyStroke k)`
  sets the keystroke `k` as accelerator for this menu item. The accelerator key is displayed next to the label.

`javax.swing.AbstractButton` 1.2

- `void setMnemonic(int mnemonic)`
  sets the mnemonic character for the button. This character will be underlined in the label.
- `void setDisplayedMnemonicIndex(int index)` 1.4
  sets the index of the character to be underlined in the button text. Use this method if you don't want the first occurrence of the mnemonic character to be underlined.

11.5.6. Enabling and Disabling Menu Items

Occasionally, a particular menu item should be selected only in certain contexts. For example, when a document is opened in read-only mode, the Save menu item is not meaningful. Of course, we could remove the item from the menu with the `JMenu.remove` method, but users would react with some surprise to menus whose content keeps changing. Instead, it is better to deactivate the menu items that lead to temporarily inappropriate commands. A deactivated menu item is shown in gray and cannot be selected (see **Figure 11.21**).

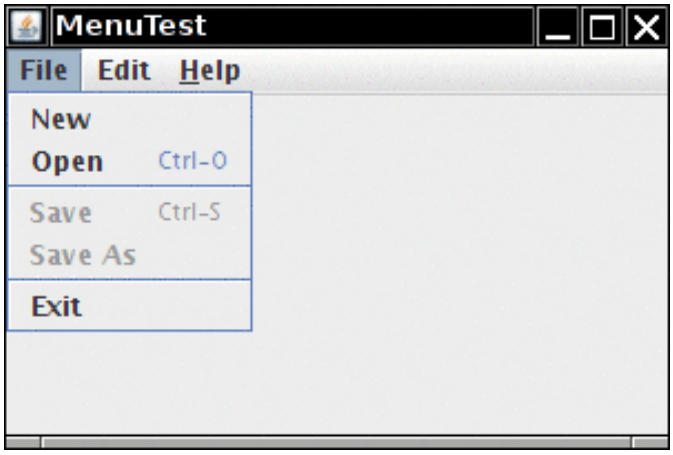


Figure 11.21: Disabled menu items

To enable or disable a menu item, use the `setEnabled` method:

```
saveItem.setEnabled(false);
```

There are two strategies for enabling and disabling menu items. Each time circumstances change, you can call `setEnabled` on the relevant menu items or actions. For example, as soon as a document has been set to read-only mode, you can locate the Save and Save As menu items and disable them. Alternatively, you can disable items just before displaying the menu. To do this, you must register a listener for the "menu selected" event. The `javax.swing.event` package defines a `MenuListener` interface with three methods:

```
void menuSelected(MenuEvent event)
void menuDeselected(MenuEvent event)
void menuCanceled(MenuEvent event)
```

The `menuSelected` method is called *before* the menu is displayed. It can therefore be used to disable or enable menu items. The following code shows how to disable the Save and Save As actions whenever the Read Only checkbox menu item is selected:

```
public void menuSelected(MenuEvent event) {
    saveAction.setEnabled(!readonlyItem.isSelected());
    saveAsAction.setEnabled(!readonlyItem.isSelected());
}
```



Caution: Disabling menu items just before displaying the menu is a clever idea, but it does not work for menu items that also have accelerator keys. Since the menu is never opened when the accelerator key is pressed, the action is never disabled, and is still triggered by the accelerator key.

`javax.swing.JMenuItem` 1.2

- `void setEnabled(boolean b)`
  enables or disables the menu item.

javax.swing.event.MenuListener 1.2

- `void menuSelected(MenuEvent e)`
  is called when the menu has been selected, before it is opened.
- `void menuDeselected(MenuEvent e)`
  is called when the menu has been deselected, after it has been closed.
- `void menuCanceled(MenuEvent e)`
  is called when the menu has been canceled, for example, by a user clicking outside the menu.

Listing 11.6 is a sample program that generates a set of menus. It shows all the features that you saw in this section: nested menus, disabled menu items, checkbox and radio button menu items, a pop-up menu, and keyboard mnemonics and accelerators.

Listing 11.6 v2ch11/menu/MenuFrame.java

```
1  package v2ch11.menu;
2
3  import module java.desktop;
4
5  /**
6   * A frame with a sample menu bar.
7   */
8  public class MenuFrame extends JFrame {
9      private static final int DEFAULT_WIDTH = 300;
10     private static final int DEFAULT_HEIGHT = 200;
11     private Action saveAction;
12     private Action saveAsAction;
13     private JCheckBoxMenuItem readonlyItem;
14
15     /**
16      * A sample action that prints the action name to System.out.
17      */
18     class TestAction extends AbstractAction {
19         public TestAction(String name) {
```

```
            super(name);
        }

        public void actionPerformed(ActionEvent event) {
            System.out.println(getValue(Action.NAME) + " selected.");
        }
    }

    public MenuFrame() {
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        var fileMenu = new JMenu("File");
        fileMenu.add(new TestAction("New"));

        // demonstrate accelerators

        var openItem = fileMenu.add(new TestAction("Open"));
        openItem.setAccelerator(KeyStroke.getKeyStroke("ctrl O"));

        fileMenu.addSeparator();

        saveAction = new TestAction("Save");
        JMenuItem saveItem = fileMenu.add(saveAction);
        saveItem.setAccelerator(KeyStroke.getKeyStroke("ctrl S"));

        saveAsAction = new TestAction("Save As");
        fileMenu.add(saveAsAction);
        fileMenu.addSeparator();

        fileMenu.add(new AbstractAction("Exit") {
            public void actionPerformed(ActionEvent event) {
                System.exit(0);
            }
        });

        // demonstrate checkbox and radio button menus

        readonlyItem = new JCheckBoxMenuItem("Read-only");
        readonlyItem.addActionListener(_ -> {
            boolean saveOk = !readonlyItem.isSelected();
            saveAction.setEnabled(saveOk);
            saveAsAction.setEnabled(saveOk);
        });

        var group = new ButtonGroup();

        var insertItem = new JRadioButtonMenuItem("Insert");
        insertItem.setSelected(true);
        var overtypeItem = new JRadioButtonMenuItem("Overtype");

        group.add(insertItem);
        group.add(overtypeItem);

        // demonstrate icons

        var cutAction = new TestAction("Cut");
        cutAction.putValue(Action.SMALL_ICON, new ImageIcon("images/cut.gif"));
        var copyAction = new TestAction("Copy");
```

```
        copyAction.putValue(Action.SMALL_ICON, new ImageIcon("images/copy.gif"));
        var pasteAction = new TestAction("Paste");
        pasteAction.putValue(Action.SMALL_ICON, new ImageIcon("images/paste.gif"));

        var editMenu = new JMenu("Edit");
        editMenu.add(cutAction);
        editMenu.add(copyAction);
        editMenu.add(pasteAction);

        // demonstrate nested menus

        var optionMenu = new JMenu("Options");

        optionMenu.add(readonlyItem);
        optionMenu.addSeparator();
        optionMenu.add(insertItem);
        optionMenu.add(overtypeItem);

        editMenu.addSeparator();
        editMenu.add(optionMenu);

        // demonstrate mnemonics

        var helpMenu = new JMenu("Help");
        helpMenu.setMnemonic('H');

        var indexItem = new JMenuItem("Index");
        indexItem.setMnemonic('I');
        helpMenu.add(indexItem);

        // you can also add the mnemonic key to an action
        var aboutAction = new TestAction("About");
        aboutAction.putValue(Action.MNEMONIC_KEY, Integer.valueOf('A'));
        helpMenu.add(aboutAction);

        // add all top-level menus to menu bar

        var menuBar = new JMenuBar();
        setJMenuBar(menuBar);

        menuBar.add(fileMenu);
        menuBar.add(editMenu);
        menuBar.add(helpMenu);

        // demonstrate pop-ups

        JPopupMenu popup = new JPopupMenu();
        popup.add(cutAction);
        popup.add(copyAction);
        popup.add(pasteAction);

        var panel = new JPanel();
        panel.setComponentPopupMenu(popup);
        add(panel);
    }
}
```

11.5.7. Toolbars

A toolbar is a button bar that gives quick access to the most commonly used commands in a program (see Figure 11.22).

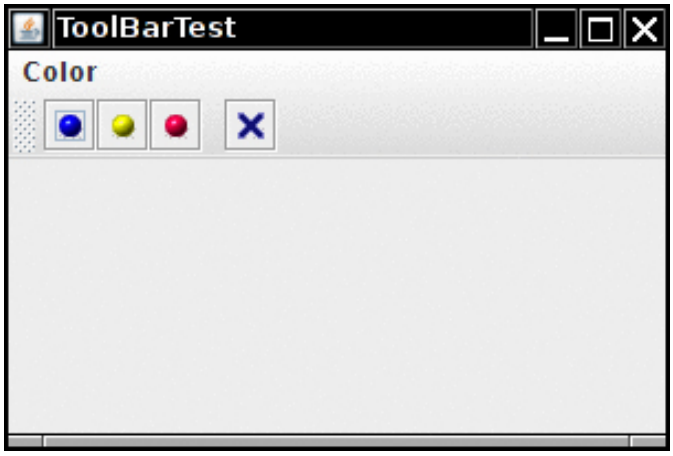


Figure 11.22: A toolbar

What makes toolbars special is that you can move them elsewhere. You can drag the toolbar to one of the four borders of the frame (see Figure 11.23). When you release the mouse button, the toolbar is dropped into the new location (see Figure 11.24).

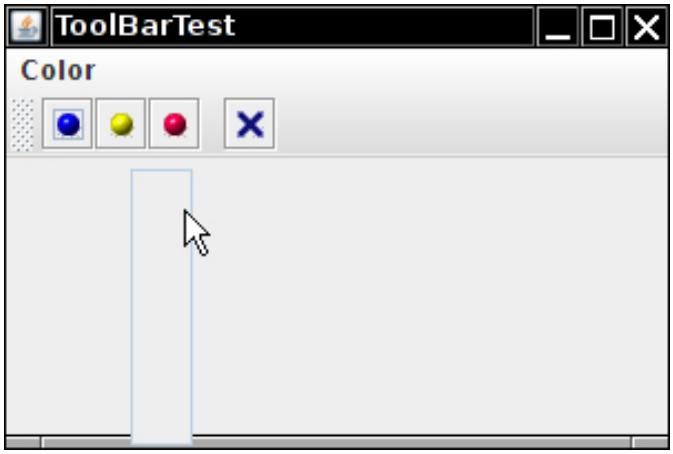


Figure 11.23: Dragging the toolbar

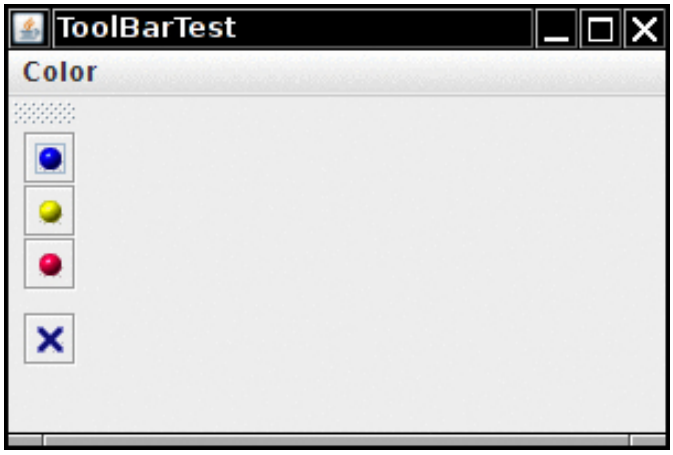


Figure 11.24: The toolbar has been dragged to another border

Note: Toolbar dragging works if the toolbar is inside a container with a border layout, or any other layout manager that supports the `North`, `East`, `South`, and `West` constraints.

The toolbar can even be completely detached from the frame. A detached toolbar is contained in its own frame (see **Figure 11.25**). When you close the frame containing a detached toolbar, the toolbar jumps back into the original frame.

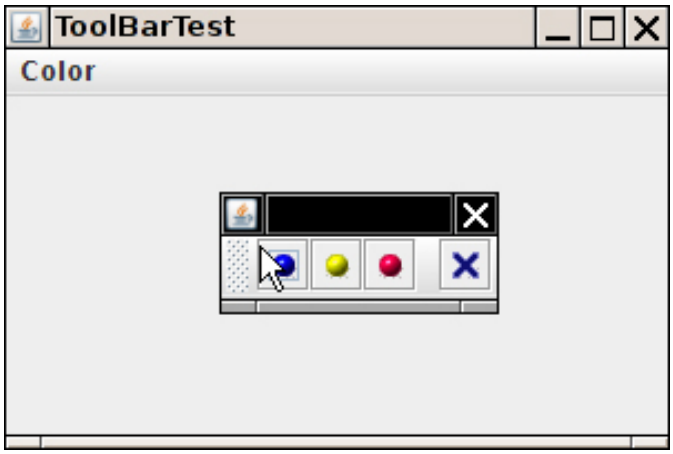


Figure 11.25: Detaching the toolbar

Toolbars are straightforward to program. Add components into the toolbar:

```
var toolbar = new JToolBar();
toolbar.add(blueButton);
```

The `JToolBar` class also has a method to add an `Action` object. Simply populate the toolbar with `Action` objects, like this:

```
toolbar.add(blueAction);
```

The small icon of the action is displayed in the toolbar.

You can separate groups of buttons with a separator:

```
toolbar.addSeparator();
```

For example, the toolbar in **Figure 11.22** has a separator between the third and fourth buttons.

Then, add the toolbar to the frame:

```
add(toolbar, BorderLayout.NORTH);
```

You can also specify a title for the toolbar that appears when the toolbar is undocked:

```
toolbar = new JToolBar(titleString);
```

By default, toolbars are initially horizontal. To have a toolbar start out vertical, use

```
toolbar = new JToolBar(SwingConstants.VERTICAL)
```

or

```
toolbar = new JToolBar(titleString, SwingConstants.VERTICAL)
```

Buttons are the most common components inside toolbars. But there is no restriction on the components that you can add to a toolbar. For example, you can add a combo box to a toolbar.

11.5.8. Tooltips

A disadvantage of toolbars is that users are often mystified by the meanings of the tiny icons in toolbars. To solve this problem, user interface designers invented *tooltips*. A tooltip is activated when the cursor rests for a moment over a button. The tooltip text is displayed inside a colored rectangle. When the user moves the mouse away, the tooltip disappears. (See **Figure 11.26**.)

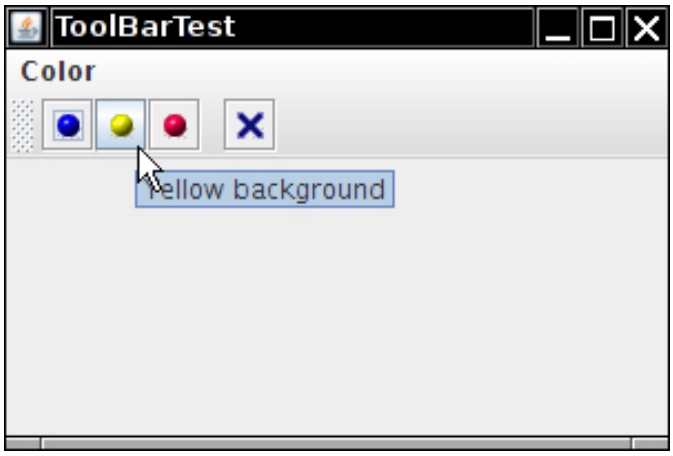


Figure 11.26: A tooltip

In Swing, you can add tooltips to any `JComponent` simply by calling the `setToolTipText` method:

```
exitButton.setToolTipText("Exit");
```

Alternatively, if you use `Action` objects, you associate the tooltip with the `SHORT_DESCRIPTION` key:

```
exitAction.putValue(Action.SHORT_DESCRIPTION, "Exit");
```

`javax.swing.JToolBar` 1.2

- `JToolBar()`
- `JToolBar(String titleString)`
- `JToolBar(int orientation)`
- `JToolBar(String titleString, int orientation)`
  construct a toolbar with the given title string and orientation. `orientation` is one of `SwingConstants.HORIZONTAL` (the default) or `SwingConstants.VERTICAL`.
- `JButton add(Action a)`
  constructs a new button inside the toolbar with name, icon, short description, and action callback from the given action, and adds the button to the end of the toolbar.
- `void addSeparator()`
  adds a separator to the end of the toolbar.

`javax.swing.JComponent` 1.2

- `void setToolTipText(String text)`
  sets the text that should be displayed as a tooltip when the mouse hovers over the component.

11.6. The Grid Bag Layout

So far we've been using only the border layout, flow layout, and grid layout for the user interface of our sample applications. For more complex tasks, this is not going to be enough.

Since Java 1.0, the AWT includes the *grid bag layout* that lays out components in rows and columns. The row and column sizes are flexible, and components can span multiple rows and columns. This layout manager is very flexible, but also very complex. The mere mention of the words “grid bag layout” has been known to strike fear in the hearts of Java programmers.

In an unsuccessful attempt to design a layout manager that would free programmers from the tyranny of the grid bag layout, the Swing designers came up with the *box layout*. According to the JDK documentation of the `BoxLayout` class: “Nesting multiple panels with different combinations of horizontal and vertical [*sic*] gives an effect similar to `GridBagLayout`, without the complexity.” However, as each box is laid out independently, you cannot use box layouts to arrange neighboring components both horizontally and vertically.

Java 1.4 saw yet another attempt to design a replacement for the grid bag layout—the *spring layout* where you use imaginary springs to connect the components in a container. As the container is resized, the springs stretch or shrink, thereby adjusting the positions of the components. This sounds tedious and confusing, and it is. The spring layout quickly sank into obscurity.

The NetBeans IDE combines a layout tool (called “Matisse”) and a layout manager. A user interface designer uses the tool to drop components into a container and to indicate which components should line up. The tool translates the designer’s intentions into instructions for the *group layout manager*. This is much more convenient than writing the layout management code by hand.

In the coming sections, I will cover the grid bag layout because it is commonly used and is still the easiest mechanism for programmatically producing layout code. I will show you a strategy that makes grid bag layouts relatively painless in common situations.

11.6.1. Grid Bag Basics

The grid bag layout is the mother of all layout managers. You can think of a grid bag layout as a grid layout without the limitations. In a grid bag layout, the rows and columns can have variable sizes. You can join adjacent cells to make room for larger components. (Many word processors, as well as HTML, provide similar capabilities for tables: You can start out with a grid and then merge adjacent cells as necessary.) The components need not fill the entire cell area, and you can specify their alignment within cells.

Consider the font selector of **Figure 11.27**. It consists of the following components:

- Two combo boxes to specify the font face and size
- Labels for these two combo boxes
- Two checkboxes to select bold and italic
- A text area for the sample string

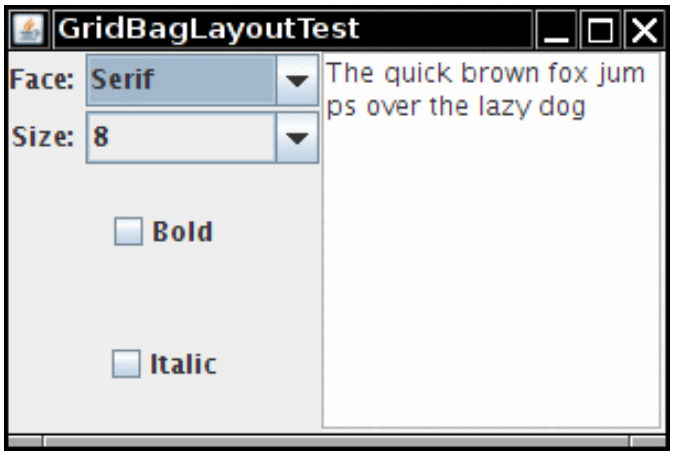


Figure 11.27: A font selector

Now, chop up the container into a grid of cells, as shown in **Figure 11.28**. (The rows and columns need not have equal size.) Each checkbox spans two columns, and the text area spans four rows.

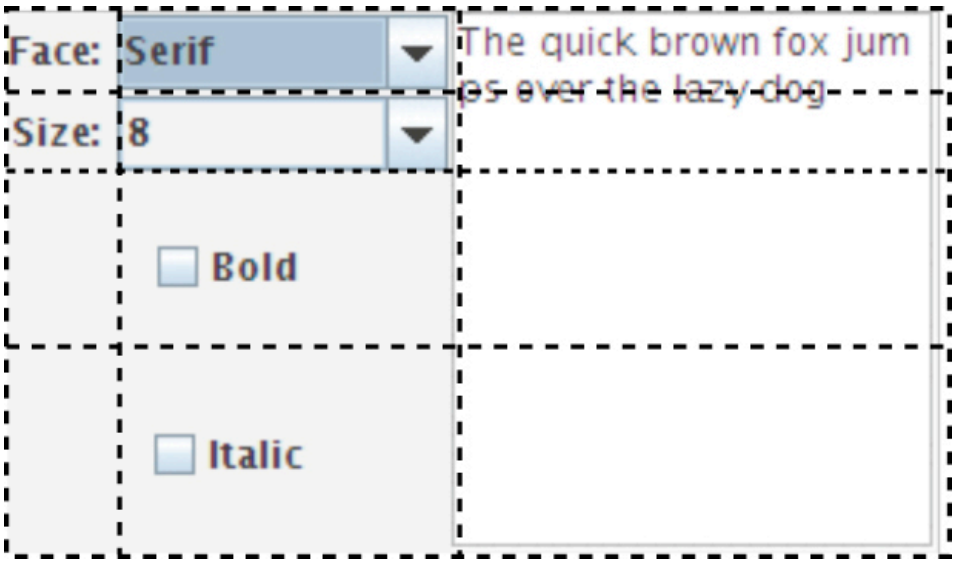


Figure 11.28: Dialog box grid used in design

To describe the layout to the grid bag manager, use the following procedure:

1. Create an object of type `GridBagLayout`. You don't need to tell it how many rows and columns the underlying grid has. Instead, the layout manager will try to guess it from the information you give it later.
2. Set this `GridBagLayout` object to be the layout manager for the component.
3. For each component, create an object of type `GridBagConstraints`. Set field values of the `GridBagConstraints` object to specify how the components are laid out within the grid bag.
4. Finally, add each component with its constraints by using the call

```
add(component, constraints);
```

Here's an example of the code needed. (We'll go over the various constraints in more detail in the sections that follow—so don't worry if you don't know what some of the constraints do.)

```
var layout = new GridBagLayout();
panel.setLayout(layout);
var constraints = new GridBagConstraints();
```

```
constraints.weightx = 100;
constraints.weighty = 100;
constraints.gridx = 0;
constraints.gridy = 2;
constraints.gridwidth = 2;
constraints.gridheight = 1;
panel.add(component, constraints);
```

The trick is knowing how to set the state of the `GridBagConstraints` object. We'll discuss this object in the sections that follow.

11.6.2. The `gridx`, `gridy`, `gridwidth`, and `gridheight` Parameters

The `gridx`, `gridy`, `gridwidth`, and `gridheight` constraints define where the component is located in the grid. The `gridx` and `gridy` values specify the column and row positions of the upper left corner of the component to be added. The `gridwidth` and `gridheight` values determine how many columns and rows the component occupies.

The top-left corner has `gridx = 0` and `gridy = 0`. The text area in our example has `gridx = 2`, `gridy = 0` because it starts in column 2 (that is, the third column) of row 0. It has `gridwidth = 1` and `gridheight = 4` because it spans one column and four rows.

11.6.3. Weight Fields

You always need to set the *weight* fields (`weightx` and `weighty`) for each area in a grid bag layout. If you set the weight to `0`, the area never grows or shrinks beyond its initial size in that direction. In the grid bag layout for **Figure 11.27**, we set the `weightx` field of the labels to be `0`. This allows the labels to keep constant width when you resize the window. On the other hand, if you set the weights for all areas to `0`, the container will huddle in the center of its allotted area instead of stretching to fill it.

Conceptually, the problem with the weight parameters is that weights are properties of rows and columns, not individual cells. But you need to specify them for cells because the grid bag layout does not expose the rows and columns. The row and column weights are computed as the maxima of the cell weights in each row or column. Thus, if you want a row or column to stay at a fixed size, you need to set the weights of all components in it to zero.

Note that the weights don't actually give the relative sizes of the columns. They tell what proportion of the "slack" space should be allocated to each area if the container exceeds its preferred size. This isn't particularly intuitive. I recommend that you set all weights at `100`. Then, run the program and see how the layout looks. Resize the dialog to see how the rows and columns adjust. If you find that a particular row or column should not grow, set the weights of all components in it to zero. You can tinker with other weight values, but it is usually not worth the effort.

11.6.4. The `fill` and `anchor` Parameters

If you don't want a component to stretch out and fill the entire area, set the `fill` constraint. You have four possibilities for this parameter: the valid values are `GridBagConstraints.NONE`, `GridBagConstraints.HORIZONTAL`, `GridBagConstraints.VERTICAL`, and `GridBagConstraints.BOTH`.

If the component does not fill the entire area, you can specify where in the area you want it by setting the `anchor` field. The valid values are `GridBagConstraints.CENTER` (the default), `GridBagConstraints.NORTH`, `GridBagConstraints.NORTHEAST`, `GridBagConstraints.EAST`, and so on.

11.6.5. Padding

You can surround a component with additional blank space by setting the `insets` field of `GridBagConstraints`. Set the `left`, `top`, `right`, and `bottom` values of the `Insets` object to the amount of space that you want to have around the component. This is called the *external padding*.

The `ipadx` and `ipady` values set the *internal padding*. These values are added to the minimum width and height of the component. This ensures that the component does not shrink down to its minimum size.

11.6.6. Alternative Method to Specify the `gridx`, `gridy`, `gridwidth`, and `gridheight` Parameters

The AWT documentation recommends that instead of setting the `gridx` and `gridy` values to absolute positions, you set them to the constant `GridBagConstraints.RELATIVE`. Then, add the components to the grid bag layout in a standardized order, going from left to right in the first row, then moving along the next row, and so on.

You would still specify the number of rows and columns spanned, by giving the appropriate `gridheight` and `gridwidth` fields. However, if the component extends to the *last* row or column, you don't need to specify the actual number, but the constant `GridBagConstraints.REMAINDER`. This tells the layout manager that the component is the last one in its row.

This scheme does seem to work. But it sounds really goofy to hide the actual placement information from the layout manager and hope that it will rediscover it.

11.6.7. A Grid Bag Layout Recipe

In practice, the following recipe makes grid bag layouts relatively trouble-free:

1. Sketch out the component layout on a piece of paper.
2. Find a grid such that the small components are each contained in a cell and the larger components span multiple cells.
3. Label the rows and columns of your grid with 0, 1, 2, 3, . . . You can now read off the `gridx`, `gridy`, `gridwidth`, and `gridheight` values.
4. For each component, ask yourself whether it needs to fill its cell horizontally or vertically. If not, how do you want it aligned? This tells you the `fill` and `anchor` parameters.
5. Set all weights to `100`. However, if you want a particular row or column to always stay at its default size, set the `weightx` or `weighty` to `0` in all components that belong to that row or column.
6. Write the code. Carefully double-check your settings for the `GridBagConstraints`. One wrong constraint can ruin your whole layout.
7. Compile, run, and enjoy.

11.6.8. A Helper Class to Tame the Grid Bag Constraints

The most tedious aspect of the grid bag layout is writing the code that sets the constraints. Most programmers write helper functions or a small helper class for this purpose. I present such a class after the complete code for the font dialog example. This class has the following features:

- Its name is short: `GBC` instead of `GridBagConstraints`.
- It extends `GridBagConstraints`, so you can use shorter names such as `GBC.EAST` for the constants.
- Use a `GBC` object when adding a component, such as

```
add(component, new GBC(1, 2));
```

- There are two constructors to set the most common parameters: `gridx` and `gridy`, or `gridx`, `gridy`, `gridwidth`, and `gridheight`.

```
add(component, new GBC(1, 2, 1, 4));
```

- There are convenient setters for the fields that come in `x`/`y` pairs:

```
add(component, new GBC(1, 2).setWeight(100, 100));
```

- The setter methods return `this`, so you can chain them:

```
add(component, new GBC(1, 2).setAnchor(GBC.EAST).setWeight(100, 100));
```

- The `setInsets` methods construct the `Insets` object for you. To get one-pixel insets, simply call

```
add(component, new GBC(1, 2).setAnchor(GBC.EAST).setInsets(1));
```

Listing 11.7 shows the frame class for the font dialog example. The `GBC` helper class is in Listing 11.8. Here is the code that adds the components to the grid bag:

```
add(faceLabel, new GBC(0, 0).setAnchor(GBC.EAST));
add(face, new GBC(1, 0).setFill(GBC.HORIZONTAL).setWeight(100, 0).setInsets(1));
add(sizeLabel, new GBC(0, 1).setAnchor(GBC.EAST));
add(size, new GBC(1, 1).setFill(GBC.HORIZONTAL).setWeight(100, 0).setInsets(1));
add(bold, new GBC(0, 2, 2, 1).setAnchor(GBC.CENTER).setWeight(100, 100));
add(italic, new GBC(0, 3, 2, 1).setAnchor(GBC.CENTER).setWeight(100, 100));
add(sample, new GBC(2, 0, 1, 4).setFill(GBC.BOTH).setWeight(100, 100));
```

Once you understand the grid bag constraints, this kind of code is fairly easy to read and debug.

Listing 11.7 v2ch11/gridbag/FontFrame.java

```
1  package v2ch11.gridbag;
2
3  import module java.desktop;
4
5  /**
6   * A frame that uses a grid bag layout to arrange font selection components.
7   */
8  public class FontFrame extends JFrame {
9      private static final int TEXT_ROWS = 10;
10     private static final int TEXT_COLUMNS = 20;
11
12     private JComboBox<String> face;
13     private JComboBox<Integer> size;
14     private JCheckBox bold;
```

```
    private JCheckBox italic;
    private JTextArea sample;

    public FontFrame() {
        var layout = new GridBagLayout();
        setLayout(layout);

        ActionListener listener = _ -> updateSample();

        // construct components

        var faceLabel = new JLabel("Face: ");

        face = new JComboBox<>(
                new String[] { "Serif", "SansSerif", "Monospaced", "Dialog", "DialogInput" });

        face.addActionListener(listener);

        var sizeLabel = new JLabel("Size: ");

        size = new JComboBox<>(new Integer[] { 8, 10, 12, 15, 18, 24, 36, 48 });

        size.addActionListener(listener);

        bold = new JCheckBox("Bold");
        bold.addActionListener(listener);

        italic = new JCheckBox("Italic");
        italic.addActionListener(listener);

        sample = new JTextArea(TEXT_ROWS, TEXT_COLUMNS);
        sample.setText("The quick brown fox jumps over the lazy dog");
        sample.setEditable(false);
        sample.setLineWrap(true);
        sample.setBorder(BorderFactory.createEtchedBorder());

        // add components to grid, using GBC convenience class

        add(faceLabel, new GBC(0, 0).setAnchor(GBC.EAST));
        add(face, new GBC(1, 0).setFill(GBC.HORIZONTAL).setWeight(100, 0).setInsets(1));
        add(sizeLabel, new GBC(0, 1).setAnchor(GBC.EAST));
        add(size, new GBC(1, 1).setFill(GBC.HORIZONTAL).setWeight(100, 0).setInsets(1));
        add(bold, new GBC(0, 2, 2, 1).setAnchor(GBC.CENTER).setWeight(100, 100));
        add(italic, new GBC(0, 3, 2, 1).setAnchor(GBC.CENTER).setWeight(100, 100));
        add(sample, new GBC(2, 0, 1, 4).setFill(GBC.BOTH).setWeight(100, 100));
        pack();
        updateSample();
    }

    public void updateSample() {
        var fontFace = (String) face.getSelectedItem();
        int fontStyle = (bold.isSelected() ? Font.BOLD : 0)
                + (italic.isSelected() ? Font.ITALIC : 0);
        int fontSize = size.getItemAt(size.getSelectedIndex());
        var font = new Font(fontFace, fontStyle, fontSize);
        sample.setFont(font);
```

```
        sample.repaint();
    }
}
```

Listing 11.8 v2ch11/gridbag/GBC.java

```
package v2ch11.gridbag;

import module java.desktop;

/**
 * This class simplifies the use of the GridBagConstraints class.
 */
public class GBC extends GridBagConstraints {
    /**
     * Constructs a GBC with a given gridx and gridy position and all other grid bag constraint
     * values set to the default.
     * @param gridx the gridx position
     * @param gridy the gridy position
     */
    public GBC(int gridx, int gridy) {
        this.gridx = gridx;
        this.gridy = gridy;
    }

    /**
     * Constructs a GBC with given gridx, gridy, gridwidth, gridheight and all other grid bag
     * constraint values set to the default.
     * @param gridx the gridx position
     * @param gridy the gridy position
     * @param gridwidth the cell span in x-direction
     * @param gridheight the cell span in y-direction
     */
    public GBC(int gridx, int gridy, int gridwidth, int gridheight) {
        this.gridx = gridx;
        this.gridy = gridy;
        this.gridwidth = gridwidth;
        this.gridheight = gridheight;
    }

    /**
     * Sets the anchor.
     * @param anchor the anchor value
     * @return this object for further modification
     */
    public GBC setAnchor(int anchor) {
        this.anchor = anchor;
        return this;
    }

    /**
     * Sets the fill direction.
     * @param fill the fill direction
     * @return this object for further modification
     */
    public GBC setFill(int fill) {
        this.fill = fill;
```

```
        return this;
    }

    /**
     * Sets the cell weights.
     * @param weightx the cell weight in x-direction
     * @param weighty the cell weight in y-direction
     * @return this object for further modification
     */
    public GBC setWeight(double weightx, double weighty) {
        this.weightx = weightx;
        this.weighty = weighty;
        return this;
    }

    /**
     * Sets the insets of this cell.
     * @param distance the spacing to use in all directions
     * @return this object for further modification
     */
    public GBC setInsets(int distance) {
        this.insets = new Insets(distance, distance, distance, distance);
        return this;
    }

    /**
     * Sets the insets of this cell.
     * @param top the spacing to use on top
     * @param left the spacing to use to the left
     * @param bottom the spacing to use on the bottom
     * @param right the spacing to use to the right
     * @return this object for further modification
     */
    public GBC setInsets(int top, int left, int bottom, int right) {
        this.insets = new Insets(top, left, bottom, right);
        return this;
    }

    /**
     * Sets the internal padding.
     * @param ipadx the internal padding in x-direction
     * @param ipady the internal padding in y-direction
     * @return this object for further modification
     */
    public GBC setIpad(int ipadx, int ipady) {
        this.ipadx = ipadx;
        this.ipady = ipady;
        return this;
    }
}
```

`java.awt.GridBagConstraints` 1.0

- `int gridx`
  `int gridy`
  specify the starting column and row of the cell. The default is `0`.

- `int gridwidth`
  `int gridheight`
  specify the column and row extent of the cell. The default is `1`.
- `double weightx`
  `double weighty`
  specify the capacity of the cell to grow. The default is `0`.
- `int anchor`
  indicates the alignment of the component inside the cell. You can choose between absolute positions:

`NORTHWEST`	`NORTH`	`NORTHEAST`
`WEST`	`CENTER`	`EAST`
`SOUTHWEST`	`SOUTH`	`SOUTHEAST`

  or their orientation-independent counterparts:

`FIRST_LINE_START`	`LINE_START`	`FIRST_LINE_END`
`PAGE_START`	`CENTER`	`PAGE_END`
`LAST_LINE_START`	`LINE_END`	`LAST_LINE_END`

  Use the latter if your application may be localized for right-to-left or top-to-bottom text. The default is `CENTER`.
- `int fill`
  specifies the fill behavior of the component inside the cell: one of `NONE`, `BOTH`, `HORIZONTAL`, or `VERTICAL`. The default is `NONE`.
- `int ipadx`
  `int ipady`
  specify the “internal” padding around the component. The default is `0`.
- `Insets insets`
  specifies the “external” padding along the cell boundaries. The default is no padding.
- `GridBagConstraints(int gridx, int gridy, int gridwidth, int gridheight, double weightx, double weighty, int anchor, int fill, Insets insets, int ipadx, int ipady)` 1.2
  constructs a `GridBagConstraints` with all its fields specified in the parameters. This constructor should only be used by automatic code generators because it makes your source code very hard to read.

11.7. Custom Layout Managers

You can design your own `LayoutManager` class that manages components in a special way. As a fun example, let's arrange all components in a container to form a circle (see **Figure 11.29**).

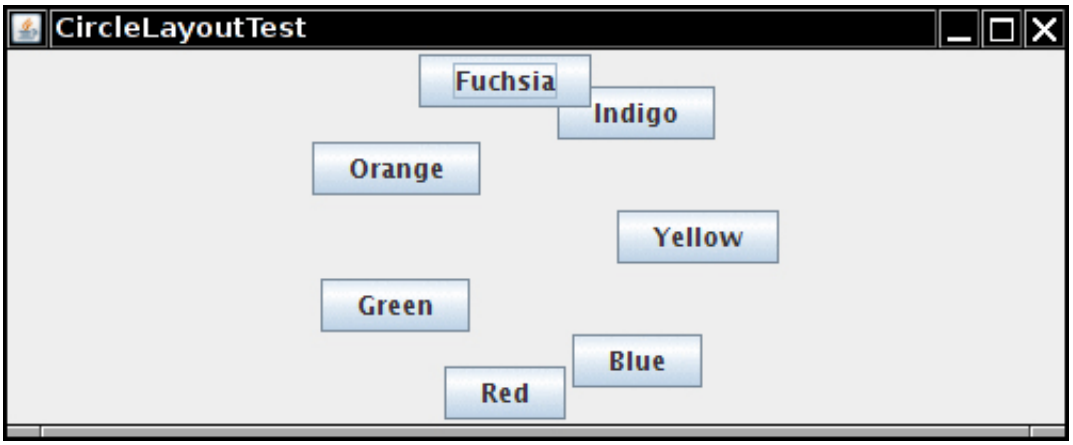


Figure 11.29: Circle layout

Your own layout manager must implement the `LayoutManager` interface. You need to override the following five methods:

```
void addLayoutComponent(String s, Component c)
void removeLayoutComponent(Component c)
Dimension preferredLayoutSize(Container parent)
Dimension minimumLayoutSize(Container parent)
void layoutContainer(Container parent)
```

The first two methods are called when a component is added or removed. If you don't keep any additional information about the components, you can make them do nothing. The next two methods compute the space required for the minimum and the preferred layout of the components. These are usually the same quantity. The fifth method does the actual work and invokes `setBounds` on all components.



Note: The AWT has a second interface, called `LayoutManager2`, with ten methods to implement rather than five. The main point of the `LayoutManager2` interface is to allow you to use the `add` method with constraints. For example, the `BorderLayout` and `GridBagLayout` implement the `LayoutManager2` interface.

Listing 11.9 shows the code for the `CircleLayout` manager which, uselessly enough, lays out the components along a circle inside the parent. The frame class of the sample program is in Listing 11.10.

Listing 11.9 v2ch11/circleLayout/CircleLayout.java

```
1  package v2ch11.circleLayout;
2
3  import module java.desktop;
4
5  /**
6   * A layout manager that lays out components along a circle.
7   */
8  public class CircleLayout implements LayoutManager {
9      private int minWidth = 0;
10     private int minHeight = 0;
11     private int preferredWidth = 0;
12     private int preferredHeight = 0;
13     private boolean sizesSet = false;
14     private int maxComponentWidth = 0;
15     private int maxComponentHeight = 0;
16
```

```
    public void addLayoutComponent(String name, Component comp) {
    }

    public void removeLayoutComponent(Component comp) {
    }

    public void setSizes(Container parent) {
        if (sizesSet) return;
        int n = parent.getComponentCount();

        preferredWidth = 0;
        preferredHeight = 0;
        minWidth = 0;
        minHeight = 0;
        maxComponentWidth = 0;
        maxComponentHeight = 0;

        // compute the maximum component widths and heights
        // and set the preferred size to the sum of the component sizes
        for (int i = 0; i < n; i++) {
            Component c = parent.getComponent(i);
            if (c.isVisible()) {
                Dimension d = c.getPreferredSize();
                maxComponentWidth = Math.max(maxComponentWidth, d.width);
                maxComponentHeight = Math.max(maxComponentHeight, d.height);
                preferredWidth += d.width;
                preferredHeight += d.height;
            }
        }
        minWidth = preferredWidth / 2;
        minHeight = preferredHeight / 2;
        sizesSet = true;
    }

    public Dimension preferredLayoutSize(Container parent) {
        setSizes(parent);
        Insets insets = parent.getInsets();
        int width = preferredWidth + insets.left + insets.right;
        int height = preferredHeight + insets.top + insets.bottom;
        return new Dimension(width, height);
    }

    public Dimension minimumLayoutSize(Container parent) {
        setSizes(parent);
        Insets insets = parent.getInsets();
        int width = minWidth + insets.left + insets.right;
        int height = minHeight + insets.top + insets.bottom;
        return new Dimension(width, height);
    }

    public void layoutContainer(Container parent) {
        setSizes(parent);

        // compute center of the circle

        Insets insets = parent.getInsets();
        int containerWidth = parent.getSize().width - insets.left - insets.right;
        int containerHeight = parent.getSize().height - insets.top - insets.bottom;
```

```
75
76          int xcenter = insets.left + containerWidth / 2;
77          int ycenter = insets.top + containerHeight / 2;
78
79          // compute radius of the circle
80
81          int xradius = (containerWidth - maxComponentWidth) / 2;
82          int yradius = (containerHeight - maxComponentHeight) / 2;
83          int radius = Math.min(xradius, yradius);
84
85          // lay out components along the circle
86
87          int n = parent.getComponentCount();
88          for (int i = 0; i < n; i++) {
89              Component c = parent.getComponent(i);
90              if (c.isVisible()) {
91                  double angle = 2 * Math.PI * i / n;
92
93                  // center point of component
94                  int x = xcenter + (int) (Math.cos(angle) * radius);
95                  int y = ycenter + (int) (Math.sin(angle) * radius);
96
97                  // move component so that its center is (x, y)
98                  // and its size is its preferred size
99                  Dimension d = c.getPreferredSize();
100                 c.setBounds(x - d.width / 2, y - d.height / 2, d.width, d.height);
101             }
102         }
103     }
104 }
```

Listing 11.10 v2ch11/circleLayout/CircleLayoutFrame.java

```
1  package v2ch11.circleLayout;
2
3  import module java.desktop;
4
5  /**
6   * A frame that shows buttons arranged along a circle.
7   */
8  public class CircleLayoutFrame extends JFrame {
9      public CircleLayoutFrame() {
10         setLayout(new CircleLayout());
11         add(new JButton("Yellow"));
12         add(new JButton("Blue"));
13         add(new JButton("Red"));
14         add(new JButton("Green"));
15         add(new JButton("Orange"));
16         add(new JButton("Fuchsia"));
17         add(new JButton("Indigo"));
18         pack();
19     }
20 }
```

java.awt.LayoutManager 1.0

- `void addLayoutComponent(String name, Component comp)`
  adds a component to the layout.
- `void removeLayoutComponent(Component comp)`
  removes a component from the layout.
- `Dimension preferredLayoutSize(Container cont)`
  returns the preferred size dimensions for the container under this layout.
- `Dimension minimumLayoutSize(Container cont)`
  returns the minimum size dimensions for the container under this layout.
- `void layoutContainer(Container cont)`
  lays out the components in a container.

11.8. Dialog Boxes

In GUI applications, you usually want separate dialog boxes to pop up to give information to, or get information from, the user.

Just as with most windowing systems, AWT distinguishes between *modal* and *modeless* dialog boxes. A modal dialog box won't let users interact with the remaining windows of the application until he or she deals with it. Use a modal dialog box when you need information from the user before you can proceed with execution. For example, when the user wants to read a file, a modal file dialog box is the one to pop up. The user must specify a file name before the program can begin the read operation. Only when the user closes the modal dialog box can the application proceed.

A modeless dialog box lets the user enter information in both the dialog box and the remainder of the application. One example of a modeless dialog is a toolbar. The toolbar can stay in place as long as needed, and the user can interact with both the application window and the toolbar.

Let's start this section with the simplest dialogs—modal dialogs with just a single message. Swing has a convenient `JOptionPane` class that lets you put up a simple dialog without writing any special dialog box code. Next, you will see how to write more complex dialogs by implementing your own dialog windows. Finally, you will see how to transfer data from your application into a dialog and back.

We'll conclude the discussion of dialog boxes by looking at the Swing `JFileChooser`.

11.8.1. Option Panes

Swing has a set of ready-made simple dialogs that suffice to ask the user for a single piece of information. The `JOptionPane` has four static methods to show these simple dialogs:

`showMessageDialog`	Show a message and wait for the user to click OK
`showConfirmDialog`	Show a message and get a confirmation (like OK/Cancel)
`showOptionDialog`	Show a message and get a user option (typically from a sequence of buttons)
`showInputDialog`	Show a message and get user input (typically from an input field or combo box)

Figure 11.30 shows a typical dialog. As you can see, the dialog has the following components:

- An icon
- A message
- One or more option buttons



Figure 11.30: A `JOptionPane`

The input dialog has an additional component for user input. This can be a text field into which the user can type an arbitrary string, or a combo box from which the user can select one item.

The exact layout of these dialogs and the choice of icons for standard message types depend on the pluggable look-and-feel.

The icon on the left side depends on one of five *message types*:

```
ERROR_MESSAGE
INFORMATION_MESSAGE
WARNING_MESSAGE
QUESTION_MESSAGE
PLAIN_MESSAGE
```

The `PLAIN_MESSAGE` type has no icon. Each dialog type also has a method that lets you supply your own icon instead.

For each dialog type, you can specify a message. This message can be a string, an icon, a user interface component, or any other object. Here is how the message object is displayed:

`String`	Draw the string
`Icon`	Show the icon
`Component`	Show the component
`Object[]`	Show all objects in the array, stacked on top of each other
Any other object	Apply `toString` and show the resulting string

Of course, supplying a message string is by far the most common case. Supplying a `Component` gives you ultimate flexibility because you can make the `paintComponent` method draw anything you want.

The buttons at the bottom depend on the dialog type and the *option type*. When calling `showMessageDialog` and `showInputDialog`, you get only a standard set of buttons (OK and OK/Cancel, respectively). When calling `showConfirmDialog`, you can choose among four option types:

```
DEFAULT_OPTION
YES_NO_OPTION
YES_NO_CANCEL_OPTION
OK_CANCEL_OPTION
```

With the `showOptionDialog` you can specify an arbitrary set of options. You supply an array of objects for the options. Each array element is rendered as follows:

`String`	Make a button with the string as label
`Icon`	Make a button with the icon as label
`Component`	Show the component
Any other object	Apply `toString` and make a button with the resulting string as label

The return values of these functions are as follows:

`showMessageDialog`	None
`showConfirmDialog`	An integer representing the chosen option
`showOptionDialog`	An integer representing the chosen option
`showInputDialog`	The string that the user supplied or selected

The `showConfirmDialog` and `showOptionDialog` return integers to indicate which button the user chose. For the option dialog, this is simply the index of the chosen option or the value `CLOSED_OPTION` if the user closed the dialog instead of choosing an option. For the confirmation dialog, the return value can be one of the following:

```
OK_OPTION
CANCEL_OPTION
YES_OPTION
NO_OPTION
CLOSED_OPTION
```

This all sounds like a bewildering set of choices, but in practice it is simple. Follow these steps:

1. Choose the dialog type (message, confirmation, option, or input).
2. Choose the icon (error, information, warning, question, none, or custom).
3. Choose the message (string, icon, custom component, or a stack of them).
4. For a confirmation dialog, choose the option type (default, Yes/No, Yes/No/Cancel, or OK/Cancel).
5. For an option dialog, choose the options (strings, icons, or custom components) and the default option.
6. For an input dialog, choose between a text field and a combo box.
7. Locate the appropriate method to call in the `JOptionPane` API.

For example, suppose you want to show the dialog in **Figure 11.30**. The dialog shows a message and asks the user to confirm or cancel. Thus, it is a confirmation dialog. The icon is a question icon. The message is a string. The option type is `OK_CANCEL_OPTION`. Here is the call you would make:

```
int selection = JOptionPane.showConfirmDialog(parent,
    "Message", "Title",
    JOptionPane.OK_CANCEL_OPTION,
    JOptionPane.QUESTION_MESSAGE);
if (selection == JOptionPane.OK_OPTION) . . .
```



Tip: The message string can contain newline (`'\n'`) characters. Such a string is displayed in multiple lines.

`javax.swing.JOptionPane` 1.2

- `static void showMessageDialog(Component parent, Object message, String title, int messageType, Icon icon)`
- `static void showMessageDialog(Component parent, Object message, String title, int messageType)`
- `static void showMessageDialog(Component parent, Object message)`
- `static void showInternalMessageDialog(Component parent, Object message, String title, int messageType, Icon icon)`
- `static void showInternalMessageDialog(Component parent, Object message, String title, int messageType)`
- `static void showInternalMessageDialog(Component parent, Object message)`

  show a message dialog or an internal message dialog. (An internal dialog is rendered entirely within its owner's frame.) The parent component can be `null`. The message to show on the dialog can be a string, icon, component, or an array of them. The `messageType` parameter is one of `ERROR_MESSAGE`, `INFORMATION_MESSAGE`, `WARNING_MESSAGE`, `QUESTION_MESSAGE`, or `PLAIN_MESSAGE`.
- `static int showConfirmDialog(Component parent, Object message, String title, int optionType, int messageType, Icon icon)`
- `static int showConfirmDialog(Component parent, Object message, String title, int optionType, int messageType)`
- `static int showConfirmDialog(Component parent, Object message, String title, int optionType)`
- `static int showConfirmDialog(Component parent, Object message)`
- `static int showInternalConfirmDialog(Component parent, Object message, String title, int optionType, int messageType, Icon icon)`
- `static int showInternalConfirmDialog(Component parent, Object message, String title, int optionType, int messageType)`
- `static int showInternalConfirmDialog(Component parent, Object message, String title, int optionType)`
- `static int showInternalConfirmDialog(Component parent, Object message)`

  show a confirmation dialog or an internal confirmation dialog. (An internal dialog is rendered entirely within its owner's frame.) Returns the option selected by the user (one of `OK_OPTION`, `CANCEL_OPTION`, `YES_OPTION`, `NO_OPTION`), or `CLOSED_OPTION` if the user closed the dialog. The parent component can be `null`. The message to show on the dialog can be a string, icon, component, or an array of them. The `messageType` parameter is one of `ERROR_MESSAGE`, `INFORMATION_MESSAGE`, `WARNING_MESSAGE`, `QUESTION_MESSAGE`, or `PLAIN_MESSAGE`, and `optionType` is one of `DEFAULT_OPTION`, `YES_NO_OPTION`, `YES_NO_CANCEL_OPTION`, or `OK_CANCEL_OPTION`.

- `static int showOptionDialog(Component parent, Object message, String title, int optionType, int messageType, Icon icon, Object[] options, Object default)`
- `static int showInternalOptionDialog(Component parent, Object message, String title, int optionType, int messageType, Icon icon, Object[] options, Object default)`

  show an option dialog or an internal option dialog. (An internal dialog is rendered entirely within its owner's frame.) Returns the index of the option selected by the user, or `CLOSED_OPTION` if the user canceled the dialog. The parent component can be `null`. The message to show on the dialog can be a string, icon, component, or an array of them. The `messageType` parameter is one of `ERROR_MESSAGE`, `INFORMATION_MESSAGE`, `WARNING_MESSAGE`, `QUESTION_MESSAGE`, or `PLAIN_MESSAGE`, and `optionType` is one of `DEFAULT_OPTION`, `YES_NO_OPTION`, `YES_NO_CANCEL_OPTION`, or `OK_CANCEL_OPTION`. The `options` parameter is an array of strings, icons, or components.
- `static Object showInputDialog(Component parent, Object message, String title, int messageType, Icon icon, Object[] values, Object default)`
- `static String showInputDialog(Component parent, Object message, String title, int messageType)`
- `static String showInputDialog(Component parent, Object message)`
- `static String showInputDialog(Object message)`
- `static String showInputDialog(Component parent, Object message, Object default)` 1.4
- `static String showInputDialog(Object message, Object default)` 1.4
- `static Object showInternalInputDialog(Component parent, Object message, String title, int messageType, Icon icon, Object[] values, Object default)`
- `static String showInternalInputDialog(Component parent, Object message, String title, int messageType)`
- `static String showInternalInputDialog(Component parent, Object message)`

  show an input dialog or an internal input dialog. (An internal dialog is rendered entirely within its owner's frame.) Returns the input string typed by the user, or `null` if the user canceled the dialog. The parent component can be `null`. The message to show on the dialog can be a string, icon, component, or an array of them. The `messageType` parameter is one of `ERROR_MESSAGE`, `INFORMATION_MESSAGE`, `WARNING_MESSAGE`, `QUESTION_MESSAGE`, or `PLAIN_MESSAGE`.

11.8.2. Creating Dialogs

In the last section, you saw how to use the `JOptionPane` class to show a simple dialog. In this section, you will see how to create such a dialog by hand.

Figure 11.31 shows a typical modal dialog box—a program information box that is displayed when the user clicks the About button.

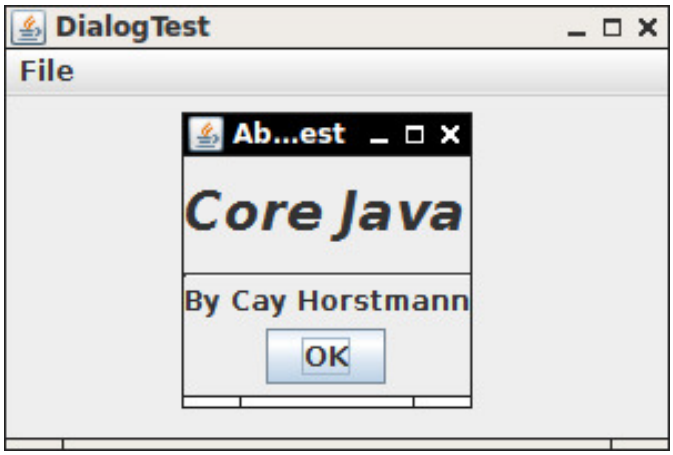


Figure 11.31: An About dialog box

To implement a dialog box, you extend the `JDialog` class. This is essentially the same process as extending `JFrame` for the main window for an application. More precisely:

1. In the constructor of your dialog box, call the constructor of the superclass `JDialog`.
2. Add the user interface components of the dialog box.
3. Add the event handlers.
4. Set the size for the dialog box.

When you call the superclass constructor, you will need to supply the *owner frame*, the title of the dialog, and the *modality*.

The owner frame controls where the dialog is displayed. You can supply `null` as the owner; then, the dialog is owned by a hidden frame.

The modality specifies which other windows of your application are blocked while the dialog is displayed. A modeless dialog does not block other windows. A modal dialog blocks all other windows of the application (except for the children of the dialog). You would use a modeless dialog for a toolbox that the user can always access. On the other hand, you would use a modal dialog if you want to force the user to supply required information before continuing.

Here's the code for a dialog box:

```
public AboutDialog extends JDialog {
    public AboutDialog(JFrame owner) {
        super(owner, "About DialogTest", true);
        add(new JLabel(
            "<html><h1><i>Core Java</i></h1><hr>By Cay Horstmann</html>"),
            BorderLayout.CENTER);
        var panel = new JPanel();
        var ok = new JButton("OK");

        ok.addActionListener(_ -> setVisible(false));
        panel.add(ok);
        add(panel, BorderLayout.SOUTH);
        setSize(250, 150);
    }
}
```

As you can see, the constructor adds user interface elements—in this case, labels and a button. It adds a handler to the button and sets the size of the dialog.

To display the dialog box, create a new dialog object and make it visible:

```
var dialog = new AboutDialog(this);
dialog.setVisible(true);
```

Actually, in the sample code below, we create the dialog box only once, and we can reuse it whenever the user clicks the About button.

```
if (dialog == null) // first time
    dialog = new AboutDialog(this);
dialog.setVisible(true);
```

When the user clicks the OK button, the dialog box should close. This is handled in the event handler of the OK button:

```
ok.addActionListener(_ -> setVisible(false));
```

When the user closes the dialog by clicking the Close button, the dialog is also hidden. Just as with a `JFrame`, you can override this behavior with the `setDefaultCloseOperation` method.

Listing 11.11 is the code for the frame class of the test program. **Listing 11.12** shows the dialog class.

Listing 11.11 v2ch11/dialog/DialogFrame.java

```
1  package v2ch11.dialog;
2
3  import module java.desktop;
4
5  /**
6   * A frame with a menu whose File->About action shows a dialog.
7   */
8  public class DialogFrame extends JFrame {
9      private static final int DEFAULT_WIDTH = 300;
10     private static final int DEFAULT_HEIGHT = 200;
11     private AboutDialog dialog;
12
13     public DialogFrame() {
14         setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
15
16         // construct a File menu
17
18         var menuBar = new JMenuBar();
19         setJMenuBar(menuBar);
20         var fileMenu = new JMenu("File");
21         menuBar.add(fileMenu);
22
23         // add About and Exit menu items
24
25         // the About item shows the About dialog
26
27         var aboutItem = new JMenuItem("About");
28         aboutItem.addActionListener(_ -> {
29             if (dialog == null) // first time
30                 dialog = new AboutDialog(DialogFrame.this);
31             dialog.setVisible(true); // pop up dialog
32         });
33         fileMenu.add(aboutItem);
34
35         // the Exit item exits the program
36
37         var exitItem = new JMenuItem("Exit");
38         exitItem.addActionListener(_ -> System.exit(0));
```

```
        fileMenu.add(exitItem);
    }
}
```

Listing 11.12 v2ch11/dialog/AboutDialog.java

```
package v2ch11.dialog;

import module java.desktop;

/**
 * A sample modal dialog that displays a message and waits for the user to click the OK button.
 */
public class AboutDialog extends JDialog {
    public AboutDialog(JFrame owner) {
        super(owner, "About DialogTest", true);

        // add HTML label to center

        add(new JLabel("<html><h1><i>Core Java</i></h1><hr>By Cay Horstmann</html>"),
                BorderLayout.CENTER);

        // OK button closes the dialog

        var ok = new JButton("OK");
        ok.addActionListener(_ -> setVisible(false));

        // add OK button to southern border

        var panel = new JPanel();
        panel.add(ok);
        add(panel, BorderLayout.SOUTH);

        pack();
    }
}
```

`javax.swing.JDialog` 1.2

- `public JDialog(Frame parent, String title, boolean modal)`
  constructs a dialog. The dialog is not visible until it is explicitly shown.

11.8.3. Data Exchange

The most common reason to put up a dialog box is to get information from the user. You have already seen how easy it is to make a dialog box object: Give it initial data and call `setVisible(true)` to display the dialog box on the screen. Now let's see how to transfer data in and out of a dialog box.

Consider the dialog box in **Figure 11.32** that could be used to obtain a username and a password to connect to some online service.

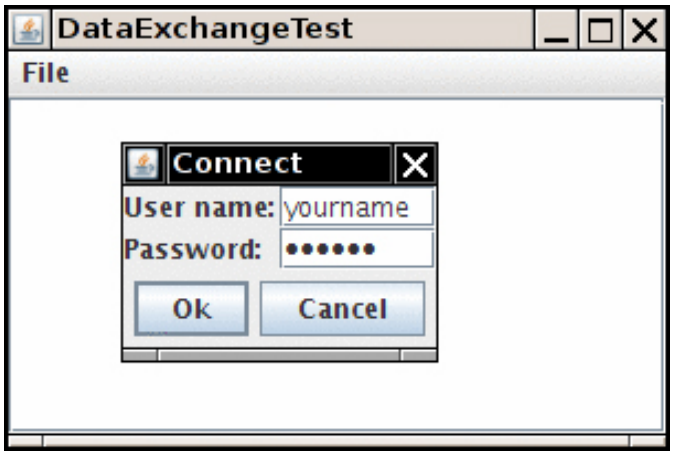


Figure 11.32: Password dialog box

Your dialog box should provide methods to set default data. For example, the `PasswordChooser` class of the example program has a method, `setUser`, to place default values into the next fields:

```
public void setUser(User u) {
    username.setText(u.getName());
}
```

Once you set the defaults (if desired), show the dialog by calling `setVisible(true)`. The dialog is now displayed.

The user then fills in the information and clicks the OK or Cancel button. The event handlers for both buttons call `setVisible(false)`, which terminates the call to `setVisible(true)`. Alternatively, the user may close the dialog. If you did not install a window listener for the dialog, the default window closing operation applies: The dialog becomes invisible, which also terminates the call to `setVisible(true)`.

The important issue is that the call to `setVisible(true)` blocks until the user has dismissed the dialog. This makes it easy to implement modal dialogs.

You want to know whether the user has accepted or canceled the dialog. Our sample code sets the `ok` flag to `false` before showing the dialog. Only the event handler for the OK button sets the `ok` flag to `true`; that's how you retrieve the user input from the dialog.



Note: Transferring data out of a modeless dialog is not as simple. When a modeless dialog is displayed, the call to `setVisible(true)` does not block and the program continues running while the dialog is displayed. If the user selects items on a modeless dialog and then clicks OK, the dialog needs to send an event to some listener in the program.

The example program contains another useful improvement. When you construct a `JDialog` object, you need to specify the owner frame. However, quite often you want to show the same dialog with different owner frames. It is better to pick the owner frame *when you are ready to show the dialog*, not when you construct the `PasswordChooser` object.

The trick is to have the `PasswordChooser` extend `JPanel` instead of `JDialog`. Build a `JDialog` object on the fly in the `showDialog` method:

```
public boolean showDialog(Frame owner, String title) {
    ok = false;
```

```
    if (dialog == null || dialog.getOwner() != owner) {
        dialog = new JDialog(owner, true);
        dialog.add(this);
        dialog.pack();
    }

    dialog.setTitle(title);
    dialog.setVisible(true);
    return ok;
}
```

Note that it is safe to have `owner` equal to `null`.

You can do even better. Sometimes, the owner frame isn't readily available. It is easy enough to compute it from any `parent` component, like this:

```
Frame owner;
if (parent instanceof Frame)
    owner = (Frame) parent;
else
    owner = (Frame) SwingUtilities.getAncestorOfClass(Frame.class, parent);
```

We use this enhancement in our sample program. The `JOptionPane` class also uses this mechanism.

Many dialogs have a *default button*, which is automatically selected if the user presses a trigger key (Enter in most look-and-feel implementations). The default button is specially marked, often with a thick outline.

Set the default button in the *root pane* of the dialog:

```
dialog.getRootPane().setDefaultButton(okButton);
```

If you follow the suggestion of laying out the dialog in a panel, then you must be careful to set the default button only after you wrapped the panel into a dialog. The panel dialog itself has no root pane.

Listing 11.13 is for the frame class of the program that illustrates the data flow into and out of a dialog box. **Listing 11.14** shows the dialog class.

Listing 11.13 v2ch11/dataExchange/DataExchangeFrame.java

```
1  package v2ch11.dataExchange;
2
3  import module java.desktop;
4
5  /**
6   * A frame with a menu whose File->Connect action shows a password dialog.
7   */
8  public class DataExchangeFrame extends JFrame {
9      public static final int TEXT_ROWS = 20;
10     public static final int TEXT_COLUMNS = 40;
11     private PasswordChooser dialog = null;
12     private JTextArea textArea;
13
```

```
    public DataExchangeFrame() {
        // construct a File menu

        var mbar = new JMenuBar();
        setJMenuBar(mbar);
        var fileMenu = new JMenu("File");
        mbar.add(fileMenu);

        // add Connect and Exit menu items

        var connectItem = new JMenuItem("Connect");
        connectItem.addActionListener(new ConnectAction());
        fileMenu.add(connectItem);

        // the Exit item exits the program

        var exitItem = new JMenuItem("Exit");
        exitItem.addActionListener(_ -> System.exit(0));
        fileMenu.add(exitItem);

        textArea = new JTextArea(TEXT_ROWS, TEXT_COLUMNS);
        add(new JScrollPane(textArea), BorderLayout.CENTER);
        pack();
    }

    /**
     * The Connect action pops up the password dialog.
     */
    private class ConnectAction implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            // if first time, construct dialog

            if (dialog == null) dialog = new PasswordChooser();

            // set default values
            dialog.setUser(new User("yourname", null));

            // pop up dialog
            if (dialog.showDialog(DataExchangeFrame.this, "Connect")) {
                // if accepted, retrieve user input
                User u = dialog.getUser();
                textArea.append("user name = " + u.getName() + ", password = "
                        + (new String(u.getPassword())) + "\n");
            }
        }
    }
}
```

Listing 11.14 v2ch11/dataExchange/PasswordChooser.java

```
package v2ch11.dataExchange;

import module java.desktop;

/**
 * A password chooser that is shown inside a dialog.
 */
```

```
public class PasswordChooser extends JPanel {
    private JTextField username;
    private JPasswordField password;
    private JButton okButton;
    private boolean ok;
    private JDialog dialog;

    public PasswordChooser() {
        setLayout(new BorderLayout());

        // construct a panel with username and password fields

        var panel = new JPanel();
        panel.setLayout(new GridLayout(2, 2));
        panel.add(new JLabel("User name:"));
        panel.add(username = new JTextField(""));
        panel.add(new JLabel("Password:"));
        panel.add(password = new JPasswordField(""));
        add(panel, BorderLayout.CENTER);

        // create Ok and Cancel buttons that terminate the dialog

        okButton = new JButton("Ok");
        okButton.addActionListener(_ -> {
            ok = true;
            dialog.setVisible(false);
        });

        var cancelButton = new JButton("Cancel");
        cancelButton.addActionListener(_ -> dialog.setVisible(false));

        // add buttons to southern border

        var buttonPanel = new JPanel();
        buttonPanel.add(okButton);
        buttonPanel.add(cancelButton);
        add(buttonPanel, BorderLayout.SOUTH);
    }

    /**
     * Sets the dialog defaults.
     * @param u the default user information
     */
    public void setUser(User u) {
        username.setText(u.getName());
    }

    /**
     * Gets the dialog entries.
     * @return a User object whose state represents the dialog entries
     */
    public User getUser() {
        return new User(username.getText(), password.getPassword());
    }

    /**
     * Show the chooser panel in a dialog.
     * @param parent a component in the owner frame or null
```

```
66       * @param title the dialog window title
67       */
68      public boolean showDialog(Component parent, String title) {
69          ok = false;
70
71          // locate the owner frame
72
73          Frame owner = (Frame) SwingUtilities.getAncestorOfClass(Frame.class, parent);
74
75          // if first time, or if owner has changed, make new dialog
76
77          if (dialog == null || dialog.getOwner() != owner) {
78              dialog = new JDialog(owner, true);
79              dialog.add(this);
80              dialog.getRootPane().setDefaultButton(okButton);
81              dialog.pack();
82          }
83
84          // set title and show dialog
85
86          dialog.setTitle(title);
87          dialog.setVisible(true);
88          return ok;
89      }
90  }
```

`javax.swing.SwingUtilities` 1.2

- `Container getAncestorOfClass(Class c, Component comp)`
  returns the innermost parent container of the given component that belongs to the given class or one of its subclasses.

`javax.swing.JComponent` 1.2

- `JRootPane getRootPane()`
  gets the root pane enclosing this component, or `null` if this component does not have an ancestor with a root pane.

`javax.swing.JRootPane` 1.2

- `void setDefaultButton(JButton button)`
  sets the default button for this root pane. To deactivate the default button, call this method with a `null` argument.

`javax.swing.JButton` 1.2

- `boolean isDefaultButton()`
  returns `true` if this button is the default button of its root pane.

11.8.4. File Dialogs

In an application, you often want to be able to open and save files. A good file dialog box that shows files and directories and lets the user navigate the file system is hard to write, and you definitely don't want to reinvent that wheel. Fortunately, Swing provides a `JFileChooser` class that allows you to display a file dialog box similar to the one that most native applications use. `JFileChooser` dialogs are always modal. Note that the `JFileChooser` class is not a subclass of `JDialog`. Instead of calling `setVisible(true)`, call `showOpenDialog` to display a dialog for opening a file, or call `showSaveDialog` to display a dialog for saving a file. The button for accepting a file is then automatically labeled Open or Save. You can also supply your own button label with the `showDialog` method. **Figure 11.33** shows an example of the file chooser dialog box.

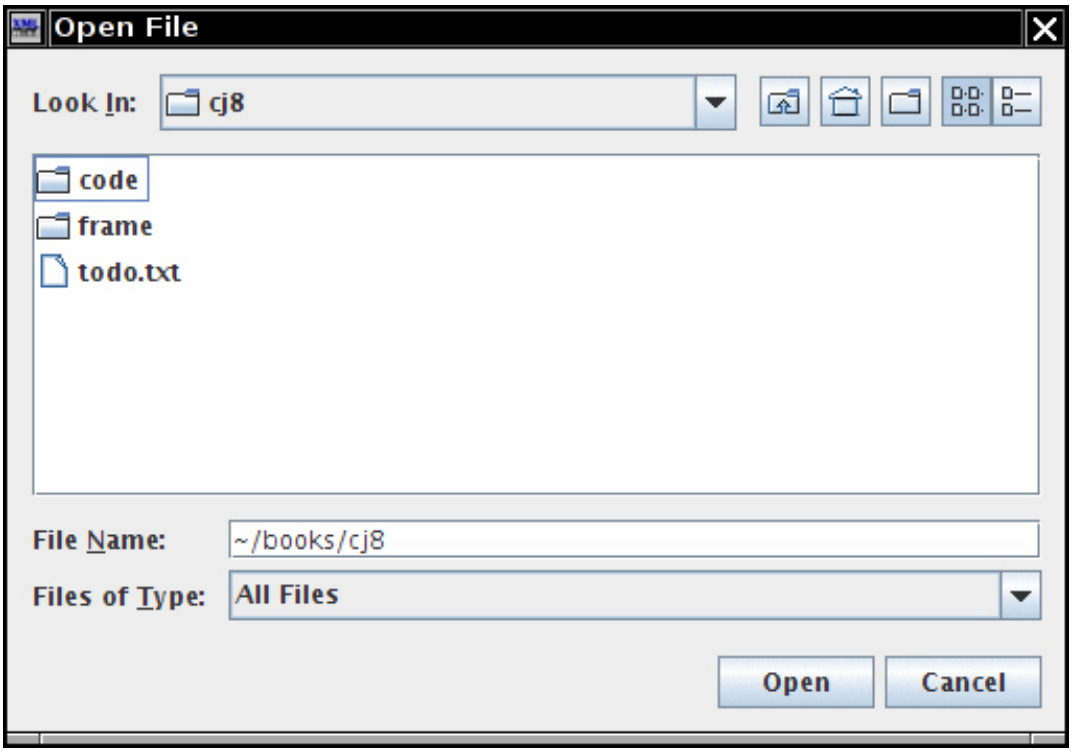


Figure 11.33: A file chooser dialog box

Here are the steps to put up a file dialog box and recover what the user chooses from the box:

1. Make a `JFileChooser` object. Unlike the constructor for the `JDialog` class, you do not supply the parent component. This allows you to reuse a file chooser dialog with multiple frames.
   For example:

   ```
   var chooser = new JFileChooser();
   ```

   Tip: Reusing a file chooser object is a good idea because the `JFileChooser` constructor can be quite slow, especially on Windows when the user has many mapped network drives.

2. Set the directory by calling the `setCurrentDirectory` method.
   For example, to use the current working directory

   ```
   chooser.setCurrentDirectory(new File("."));
   ```

   you need to supply a `File` object. `File` objects are explained in detail in **Chapter 2**. All you need to know for now is that the constructor `File(String filename)` turns a file or directory name into a `File` object.

3. If you have a default file name that you expect the user to choose, supply it with the `setSelectedFile` method:

```
chooser.setSelectedFile(new File(filename));
```

4. To enable the user to select multiple files in the dialog, call the `setMultiSelectionEnabled` method. This is, of course, entirely optional and not all that common.

```
chooser.setMultiSelectionEnabled(true);
```

5. If you want to restrict the display of files in the dialog to those of a particular type (for example, all files with extension `.gif`), you need to set a *file filter*. We discuss file filters later in this section.
6. By default, a user can select only files with a file chooser. If you want the user to select directories, use the `setFileSelectionMode` method. Call it with `JFileChooser.FILES_ONLY` (the default), `JFileChooser.DIRECTORIES_ONLY`, or `JFileChooser.FILES_AND_DIRECTORIES`.
7. Show the dialog box by calling the `showOpenDialog` or `showSaveDialog` methods. You must supply the parent component in these calls:

```
int result = chooser.showOpenDialog(parent);
```

   or

```
int result = chooser.showSaveDialog(parent);
```

   The only difference between these calls is the label of the "approve button"—the button that the user clicks to finish the file selection. You can also call the `showDialog` method and pass an explicit text for the approve button:

```
int result = chooser.showDialog(parent, "Select");
```

   These calls return only when the user has approved, canceled, or dismissed the file dialog. The return value is `JFileChooser.APPROVE_OPTION`, `JFileChooser.CANCEL_OPTION`, or `JFileChooser.ERROR_OPTION`.
8. Get the selected file or files with the `getSelectedFile()` or `getSelectedFiles()` method. These methods return either a single `File` object or an array of `File` objects. If you just need the name of the file object, call its `getPath` method. For example:

```
String filename = chooser.getSelectedFile().getPath();
```

For the most part, these steps are simple. The major difficulty with using a file dialog is to specify a subset of files from which the user should choose. For example, suppose the user should choose a GIF image file. Then, the file chooser should only display files with the extension `.gif`. It should also give the user some kind of feedback that the displayed files are of a particular category, such as "GIF Images." But the situation can be more complex. If the user should choose a JPEG image file, the extension can be either `.jpg` or `.jpeg`. Instead of a way to codify

these complexities, the designers of the file chooser provided a more elegant mechanism: to restrict the displayed files, supply an object that extends the abstract class `javax.swing.filechooser.FileFilter`. The file chooser passes each file to the file filter and displays only those files that the filter accepts.

At the time of this writing, two such subclasses are supplied: the default filter that accepts all files, and a filter that accepts all files with a given extension. However, it is easy to write ad-hoc file filters. Simply implement the two abstract methods of the `FileFilter` superclass:

```
public boolean accept(File f);
public String getDescription();
```

The first method tests whether a file should be accepted. The second method returns a description of the file type that can be displayed in the file chooser dialog.



Note: An unrelated `FileFilter` interface in the `java.io` package has a single method, `boolean accept(File f)`. It is used in the `listFiles` method of the `File` class to list files in a directory. I do not know why the designers of Swing didn't extend this interface—perhaps the Java class library had, way back in 1998, become so complex that they were no longer aware of all the standard classes and interfaces.

You will need to resolve the name conflict between these two identically named types if you import both the `java.io` and the `javax.swing.filechooser` packages. The simplest remedy is to import `javax.swing.filechooser.FileFilter`, not `javax.swing.filechooser.*`.

Once you have a file filter object, use the `setFileFilter` method of the `JFileChooser` class to install it into the file chooser object:

```
chooser.setFileFilter(new FileNameExtensionFilter("Image files", "gif", "jpg"));
```

You can install multiple filters to the file chooser by calling

```
chooser.addChoosableFileFilter(filter1);
chooser.addChoosableFileFilter(filter2);
. . .
```

The user selects a filter from the combo box at the bottom of the file dialog. By default, the "All files" filter is always present in the combo box. This is a good idea—just in case a user of your program needs to select a file with a nonstandard extension. However, if you want to suppress the "All files" filter, call

```
chooser.setAcceptAllFileFilterUsed(false);
```



Caution: If you reuse a single file chooser for loading and saving different file types, call

```
chooser.resetChoosableFilters();
```

to clear any old file filters before adding new ones.

Finally, you can customize the file chooser by providing special icons and file descriptions for each file that the file chooser displays. Do this by supplying an object of a class extending the `FileView` class in the `javax.swing.filechooser` package. This is definitely an advanced technique. Normally, you don't need to supply a file view—the pluggable look-and-feel supplies one for you. But if you want to show different icons for special file types, you can install your own file view. You need to extend the `FileView` class and implement five methods:

```
Icon getIcon(File f)
String getName(File f)
String getDescription(File f)
String getTypeDescription(File f)
Boolean isTraversable(File f)
```

Then, use the `setFileView` method to install your file view into the file chooser.

The file chooser calls your methods for each file or directory that it wants to display. If your method returns `null` for the icon, name, or description, the file chooser then consults the default file view of the look-and-feel. That is good, because it means you need to deal only with the file types for which you want to do something different.

The file chooser calls the `isTraversable` method to decide whether to open a directory when a user clicks on it. Note that this method returns a `Boolean` object, not a `boolean` value! This seems weird, but it is actually convenient—if you aren't interested in deviating from the default file view, just return `null`. The file chooser will then consult the default file view. In other words, the method returns a `Boolean` to let you choose among three options: true (`Boolean.TRUE`), false (`Boolean.FALSE`), or don't care (`null`).

The example program contains a simple file view class. That class shows a particular icon whenever a file matches a file filter. We use it to display a palette icon for all image files.

```
class FileIconView extends FileView {
    private FileFilter filter;
    private Icon icon;

    public FileIconView(FileFilter aFilter, Icon anIcon) {
        filter = aFilter;
        icon = anIcon;
    }

    public Icon getIcon(File f) {
        if (!f.isDirectory() && filter.accept(f))
            return icon;
        else return null;
    }
}
```

Install this file view into your file chooser with the `setFileView` method:

```
chooser.setFileView(new FileIconView(filter,
    new ImageIcon("palette.gif")));
```

The file chooser will then show the palette icon next to all files that pass the `filter` and use the default file view to show all other files. Naturally, we use the same filter that we set in the file chooser.

Finally, you can customize a file dialog by adding an *accessory* component. For example, **Figure 11.34** shows a preview accessory next to the file list. This accessory displays a thumbnail view of the currently selected file.

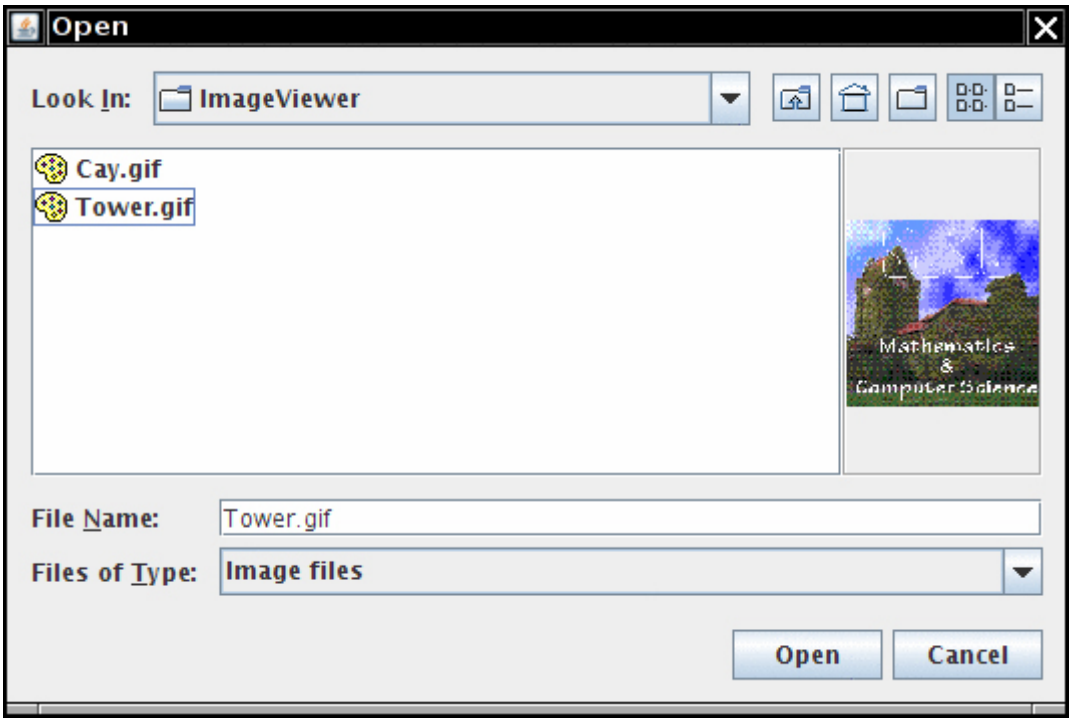


Figure 11.34: A file dialog with a preview accessory

An accessory can be any Swing component. In our case, we extend the `JLabel` class and set its icon to a scaled copy of the graphics image:

```
class ImagePreviewer extends JLabel {
    public ImagePreviewer(JFileChooser chooser) {
        setPreferredSize(new Dimension(100, 100));
        setBorder(BorderFactory.createEtchedBorder());
    }

    public void loadImage(File f) {
        var icon = new ImageIcon(f.getPath());
        if(icon.getIconWidth() > getWidth())
            icon = new ImageIcon(icon.getImage().getScaledInstance(
                getWidth(), -1, Image.SCALE_DEFAULT));
        setIcon(icon);
        repaint();
    }
}
```

There is just one challenge. We want to update the preview image whenever the user selects a different file. The file chooser uses the “JavaBeans” mechanism of notifying interested listeners whenever one of its properties changes. The selected file is a property that you can monitor by installing a `PropertyChangeListener`. Here is the code that you need to trap the notifications:

```
chooser.addPropertyChangeListener(_ -> {
    if (event.getPropertyName() == JFileChooser.SELECTED_FILE_CHANGED_PROPERTY) {
        var newFile = (File) event.getNewValue();
```

```
         // update the accessory
         . . .
      }
   });
```

`javax.swing.JFileChooser` 1.2

- `JFileChooser()`
  creates a file chooser dialog box that can be used for multiple frames.
- `void setCurrentDirectory(File dir)`
  sets the initial directory for the file dialog box.
- `void setSelectedFile(File file)`
- `void setSelectedFiles(File[] file)`
  set the default file choice for the file dialog box.
- `void setMultiSelectionEnabled(boolean b)`
  sets or clears the multiple selection mode.
- `void setFileSelectionMode(int mode)`
  lets the user select files only (the default), directories only, or both files and directories. The `mode` parameter is one of `JFileChooser.FILES_ONLY`, `JFileChooser.DIRECTORIES_ONLY`, and `JFileChooser.FILES_AND_DIRECTORIES`.
- `int showOpenDialog(Component parent)`
- `int showSaveDialog(Component parent)`
- `int showDialog(Component parent, String approveButtonText)`
  show a dialog in which the approve button is labeled "Open", "Save", or with the `approveButtonText` string. Return `APPROVE_OPTION`, `CANCEL_OPTION` (if the user selected the cancel button or dismissed the dialog), or `ERROR_OPTION` (if an error occurred).
- `File getSelectedFile()`
- `File[] getSelectedFiles()`
  get the file or files that the user selected (or return `null` if the user didn't select any file).
- `void setFileFilter(FileFilter filter)`
  sets the file mask for the file dialog box. All files for which `filter.accept` returns `true` will be displayed. Also, adds the filter to the list of choosable filters.
- `void addChoosableFileFilter(FileFilter filter)`
  adds a file filter to the list of choosable filters.
- `void setAcceptAllFileFilterUsed(boolean b)`
  includes or suppresses an "All files" filter in the filter combo box.
- `void resetChoosableFileFilters()`
  clears the list of choosable filters. Only the "All files" filter remains unless it is explicitly suppressed.
- `void setFileView(FileView view)`
  sets a file view to provide information about the files that the file chooser displays.
- `void setAccessory(JComponent component)`
  sets an accessory component.

`javax.swing.filechooser.FileFilter` 1.2

- `boolean accept(File f)`
  returns `true` if the file chooser should display this file.
- `String getDescription()`
  returns a description of this file filter—for example, `"Image files (*.gif,*.jpeg)"`.

`javax.swing.filechooser.FileNameExtensionFilter` 6

- `FileNameExtensionFilter(String description, String... extensions)`
  constructs a file filter with the given description that accepts all directories and all files whose names end in a period followed by one of the given extension strings.

`javax.swing.filechooser.FileView` 1.2

- `String getName(File f)`
  returns the name of the file `f`, or `null`. Normally, this method simply returns `f.getName()`.
- `String getDescription(File f)`
  returns a human-readable description of the file `f`, or `null`. For example, if `f` is an HTML document, this method might return its title.
- `String getTypeDescription(File f)`
  returns a human-readable description of the type of the file `f`, or `null`. For example, if `f` is an HTML document, this method might return a string `"Hypertext document"`.
- `Icon getIcon(File f)`
  returns an icon for the file `f`, or `null`. For example, if `f` is a JPEG file, this method might return a thumbnail icon.
- `Boolean isTraversable(File f)`
  returns `Boolean.TRUE` if `f` is a directory that the user can open. This method might return `Boolean.FALSE` if a directory is conceptually a compound document. Like all `FileView` methods, this method can return `null` to signify that the file chooser should consult the default view instead.

The coverage of GUI and graphics programming continues with the next chapter, which covers more advanced Swing components and sophisticated graphics techniques.

Previous chapter
< 10. Graphical User Interface Programming

Next chapter
12. Advanced Swing and Graphics >